

# Kotlin: fundamentals

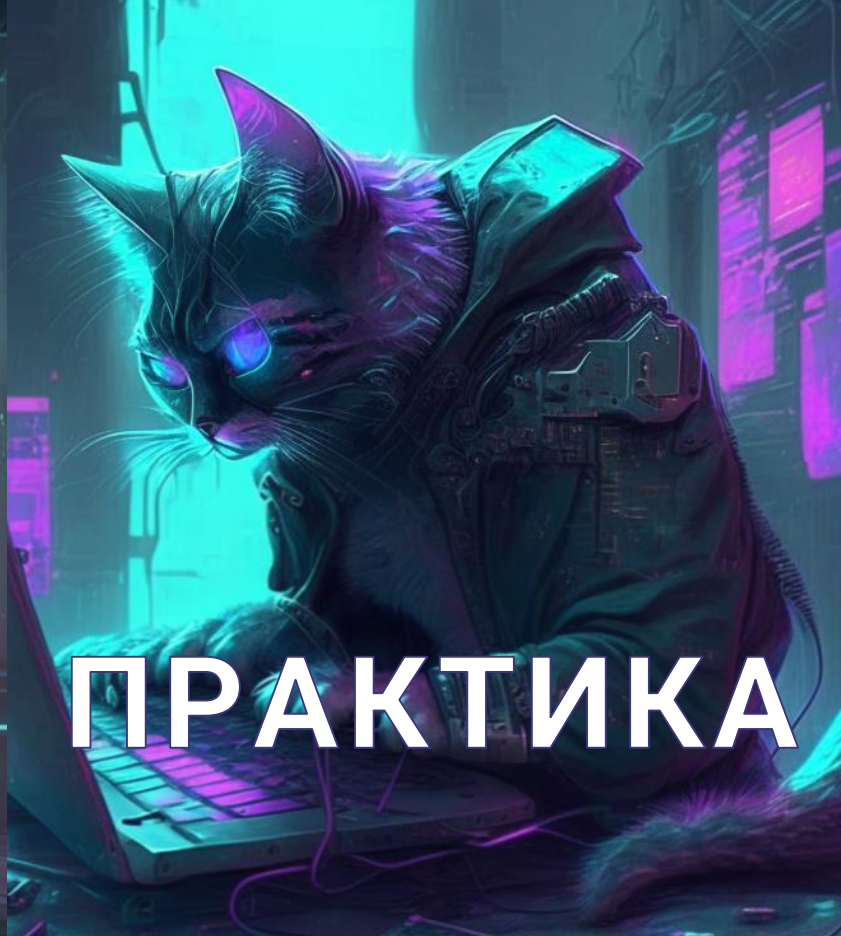
*pragmatic language*

Александр Митясов,  
специалист по технологическому обучению

nexign <academy>



ЛЕКЦИИ



ПРАКТИКА



Q&A



# План курса:

## Kotlin и основы синтаксиса

01

- ▶ особенности языка
- ▶ структура программы
- ▶ типы данных
- ▶ управление выполнением программы

## Kotlin и ООП

02

- ▶ Структура класса
- ▶ Наследование
- ▶ Интерфейсы и абстрактные классы
- ▶ Шаблоны проектирования

## Kotlin и ФП

03

- ▶ особенности языка
- ▶ структура программы
- ▶ типы данных
- ▶ управление выполнением программы

## Kotlin и основные библиотеки

04

- ▶ коллекции
- ▶ дженерики
- ▶ корутины
- ▶ DSL



# Kotlin: Есть ли жизнь после Java?



# План урока

- ▶ В предыдущих сериях
- ▶ Kotlin. Что это?
- ▶ Структура программы
- ▶ Типы данных и переменные
- ▶ Null safety
- ▶ Управление потоком программы

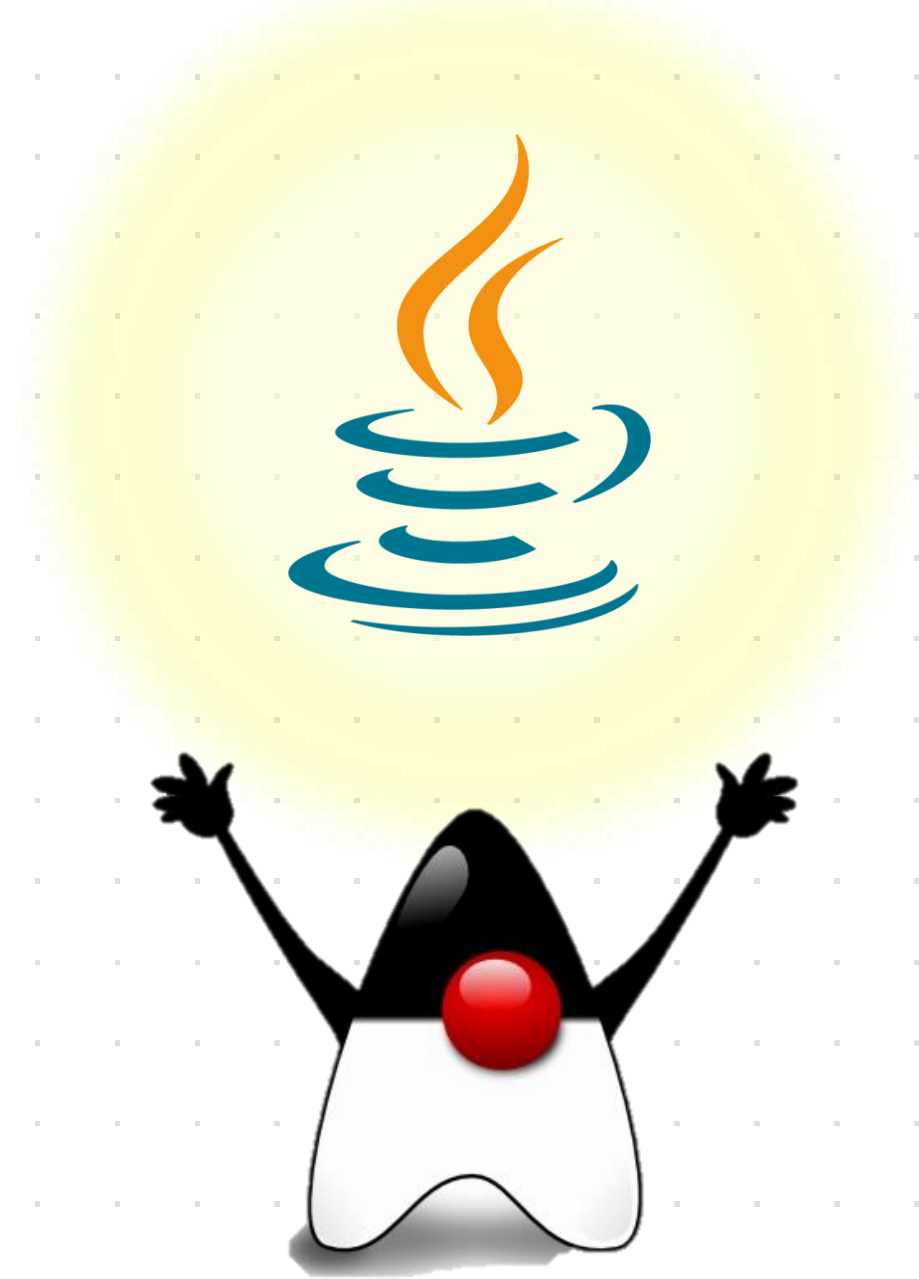
# В предыдущих сериях...

1. Что такое JVM
2. Как работает память в JVM языках
3. JDK
4. JVM – языки. Что это?



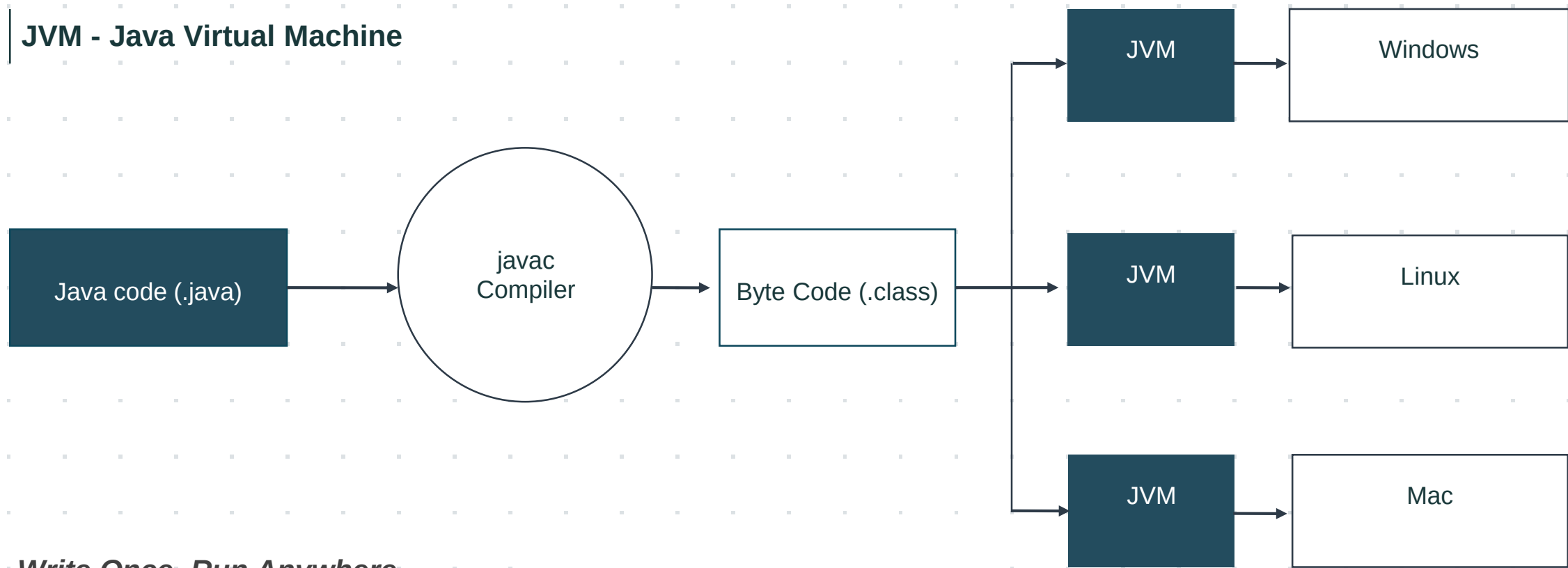
## В начале была Java...

- ▶ Переносимость (JVM)
- ▶ Объектно – ориентированность
- ▶ Автоматическое управление памятью
- ▶ Фичи заимствованные у C++



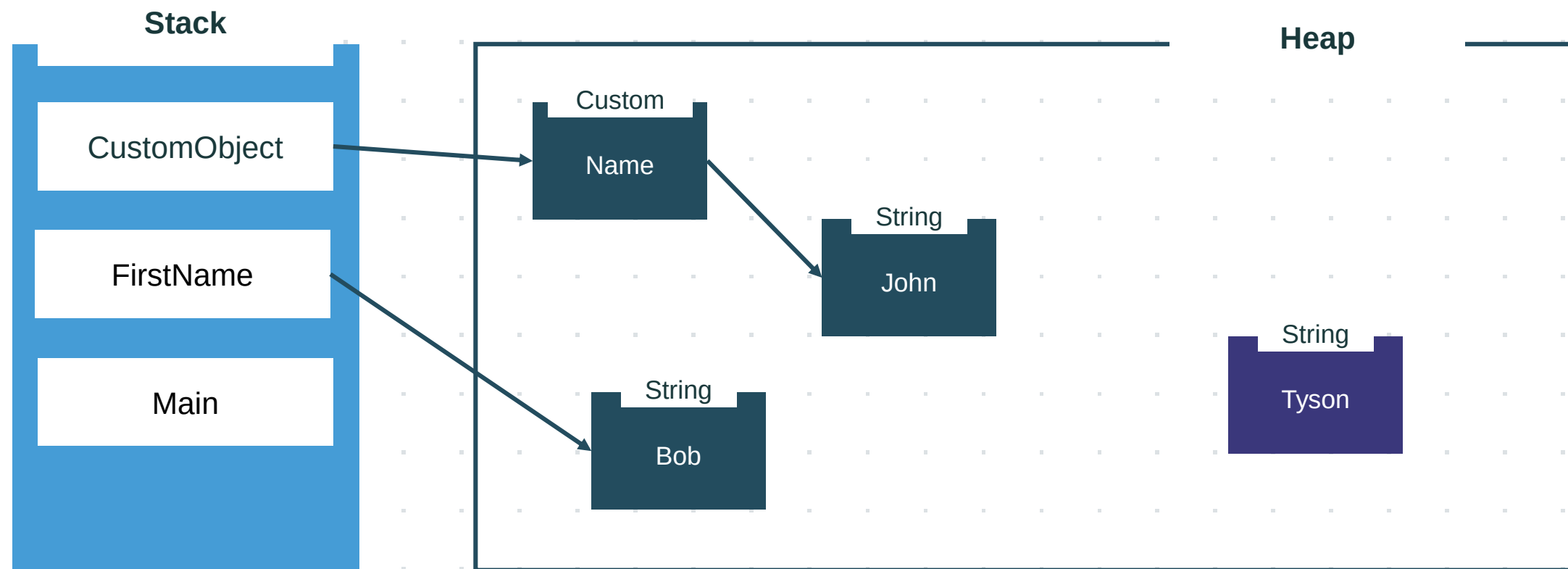
# Переносимость

## JVM - Java Virtual Machine



*Write Once, Run Anywhere*

# Управление памятью



# Инструменты

## JDK – Java Development Kit

java, javac, jar ..

### JRE – Java Runtime Enviroment

ClassLoader, ByteCode Verifier, Runtime Libraries

### JVM – Java Virtual Machine

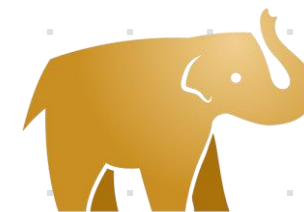
Java Interpreter, JIT, Garbage Collector

## JDK

- ▶ Oracle JDK
- ▶ OpenJDK Build от Oracle
- ▶ AdoptOpenJDK
- ▶ Red Hat OpenJDK
- ▶ Azul Zulu
- ▶ Amazon Corretto

# JVM ЯЗЫКИ

- ▶ JRuby
- ▶ Ceylon
- ▶ Groovy
- ▶ Clojure
- ▶ Kotlin
- ▶ Scala



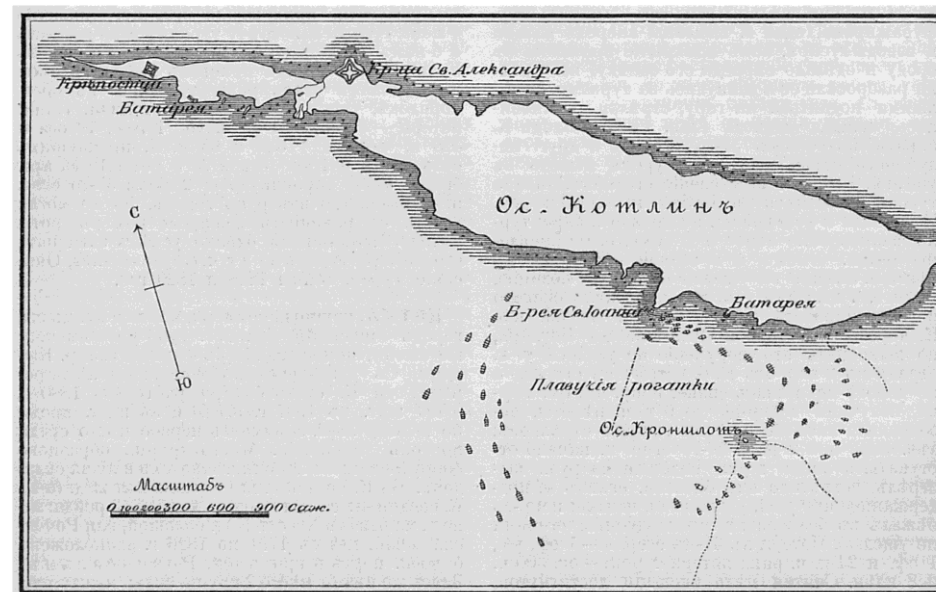
# Что такое Kotlin?

1. История
2. Предпосылки появления
3. Философия языка
4. Совместимость с Java
5. Kotlin multiplatform



# Kotlin

лаконичный язык со статической  
типизацией, безопасный и  
прагматичный. Совместим с Java.



# История появления

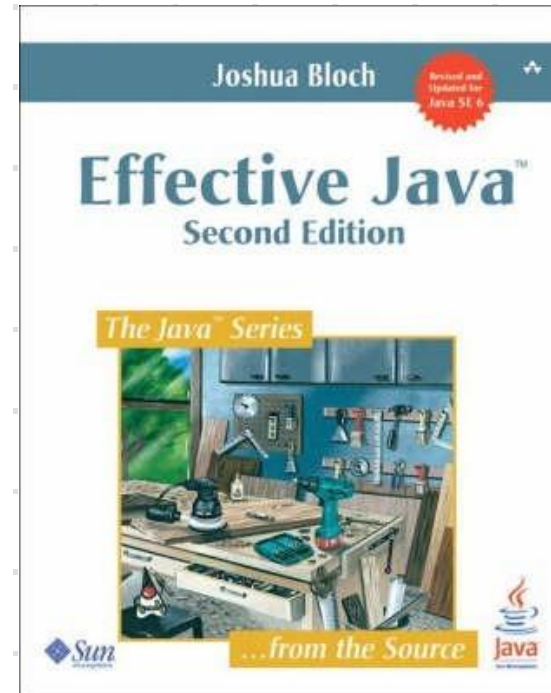
- 2010  
начало разработки внутри JetBrains
- 2012  
выложен в открытый доступ на GitHub
- 2016  
релиз версии 1.0.
- 2017  
“благословение” Google для разработки на Android



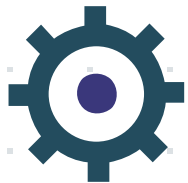
# Проблемы Java

- ▶ Отсутствие вывода типов
- ▶ Нет функций высших порядков
- ▶ Нет string interpolation
- ▶ Нет <> оператора
- ▶ Странные исключения (checked vs unchecked)
- ▶ Нет pattern matching
- ▶ Есть null
- ▶ Все очень и очень многословно

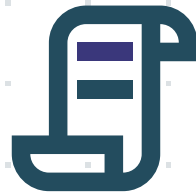
*А так же...*



# Философия Kotlin



Прагматичность



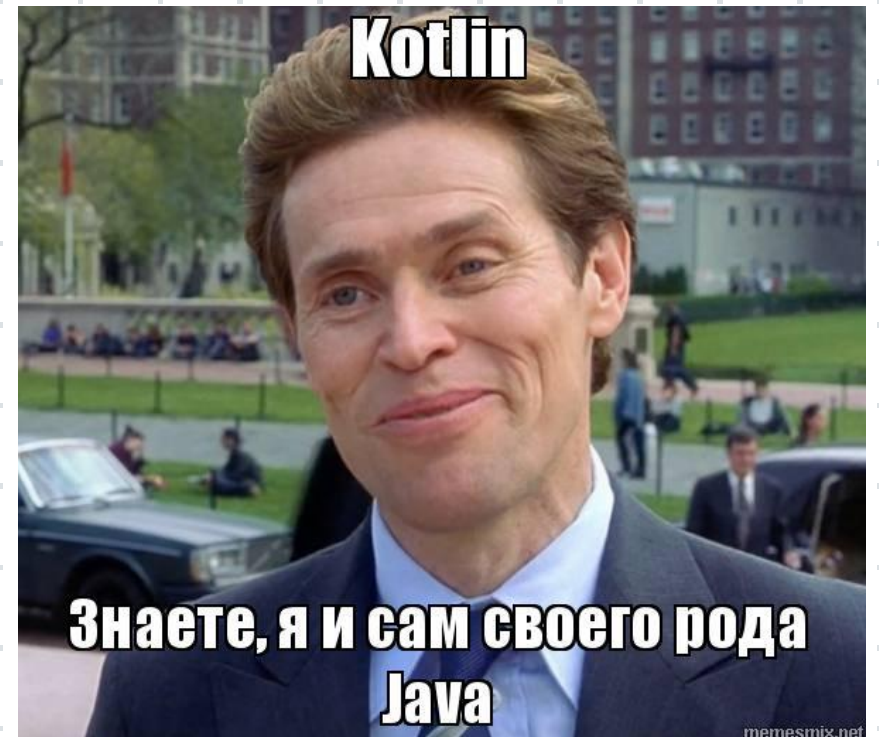
Лаконичность



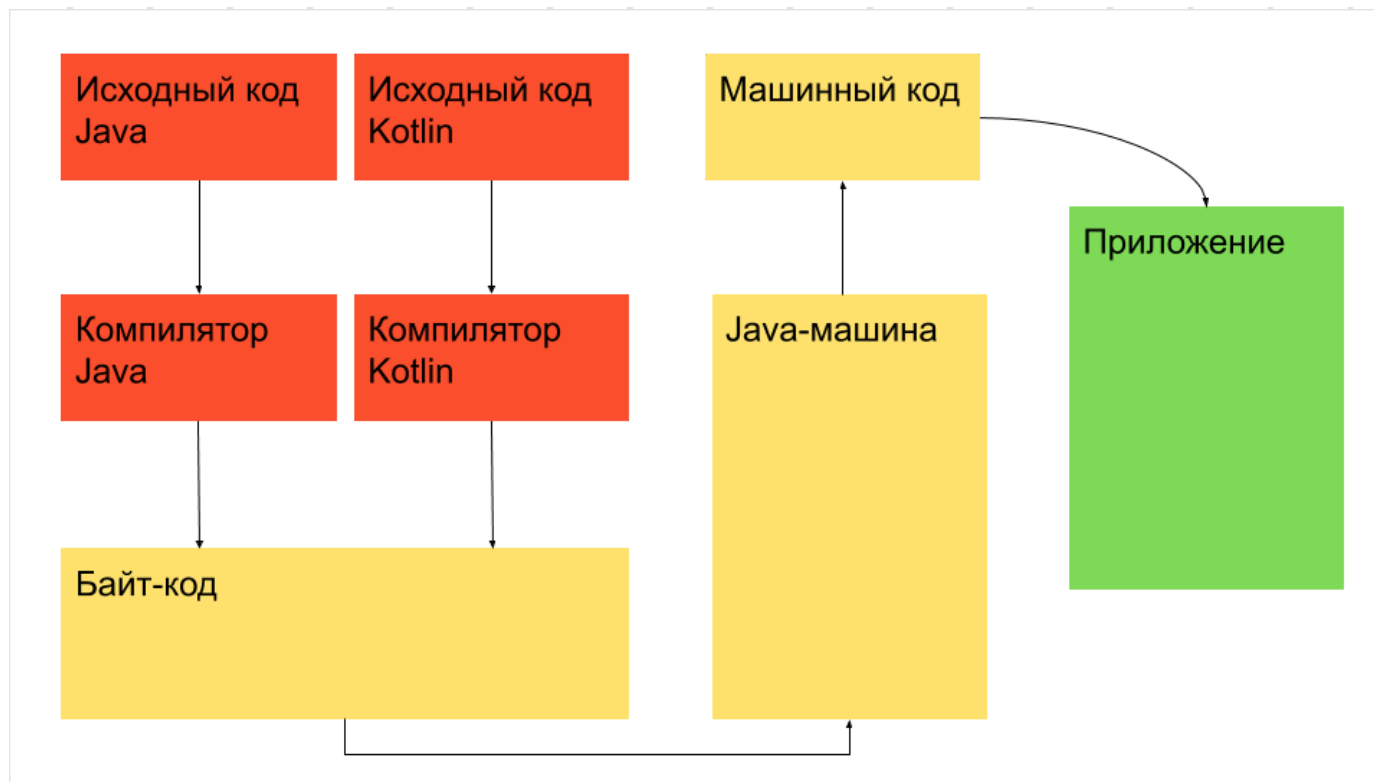
Безопасность

# Совместимость

- ▶ Классы и методы на Kotlin можно вызывать как обычные классы и методы на Java
- ▶ В Kotlin нет своей библиотеки коллекций (из Java)
- ▶ Mix Project: Java + Kotlin



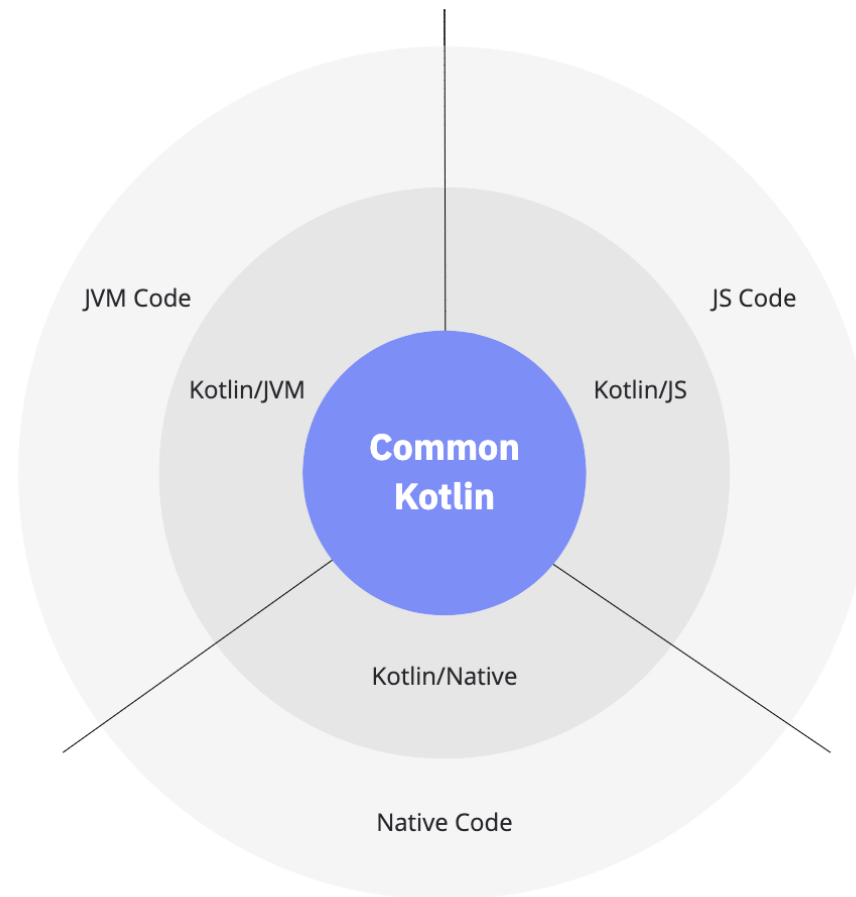
# Компиляция кода



```
kotlinc <исходный файл или каталог> -include-runtime -d <имя jar-файла>  
java -jar <имя jar-файла>
```

# Kotlin multiplatform

- ▶ Обычный Kotlin
- ▶ Kotlin для конкретной платформы (Kotlin/JVM, Kotlin/JS, Kotlin/Native)
- ▶ Доступ к встроенному коду платформы (JVM, JS и Native) и использование всех встроенных возможностей



# Practice 01 | Hi, kotlin!

1. Hello world
2. Запуск и сборка
3. Kotlin script
4. Mix project (Kotlin + Java)



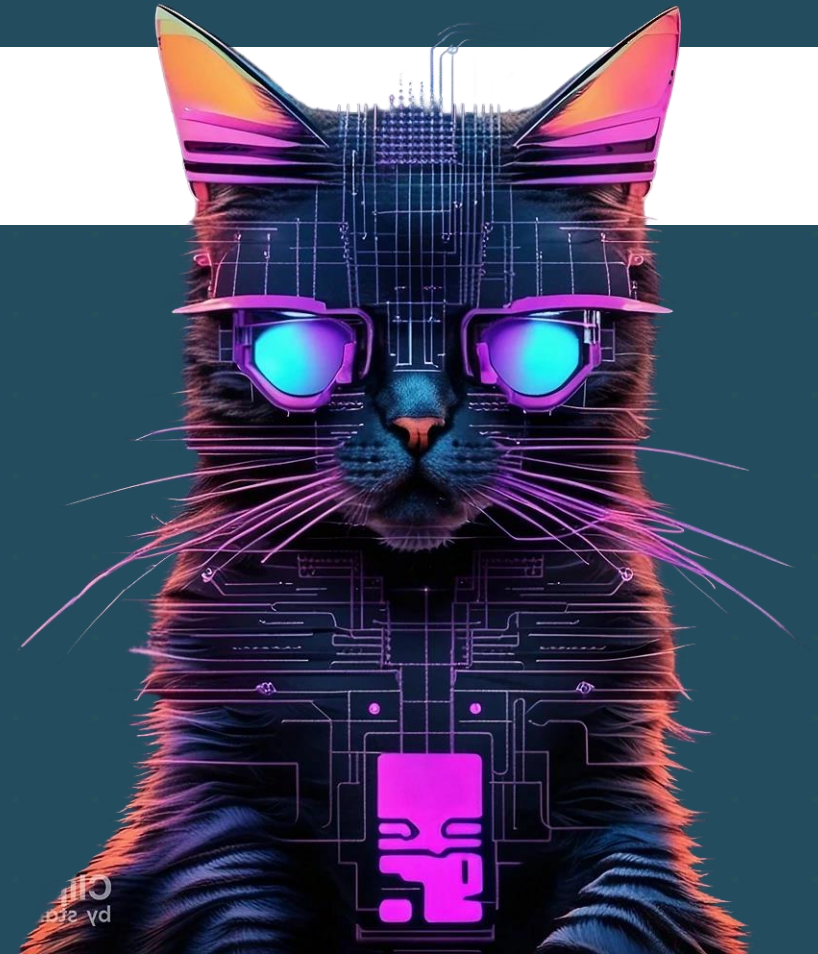
# Промежуточный итог

- ❑ Объявления функций начинаются с ключевого слова `fun`.
- ❑ Тип параметра указывается после его имени.
- ❑ Функцию можно объявить на верхнем уровне в файле – её не обязательно помещать в класс.
- ❑ `System.out.println` → `println`.
- ❑ ";" в конце строки можно опустить

```
fun main(args:Array<String>) {  
    println("Hello World!")  
}
```

# Структура программы

1. Пакеты и импорты
2. Класс или файл
3. Функция `main`
4. Рекомендации по стилю кода



# Структура программы

- ▶ Пакет
- ▶ Импорты
- ▶ Декларация класса \*
  - ▶ Члены класса
    - ▶ свойства
    - ▶ методы
  - ▶ конструкторы
- ▶ Внешние функции \*

```
package com.nexign.kotlinDemo
```

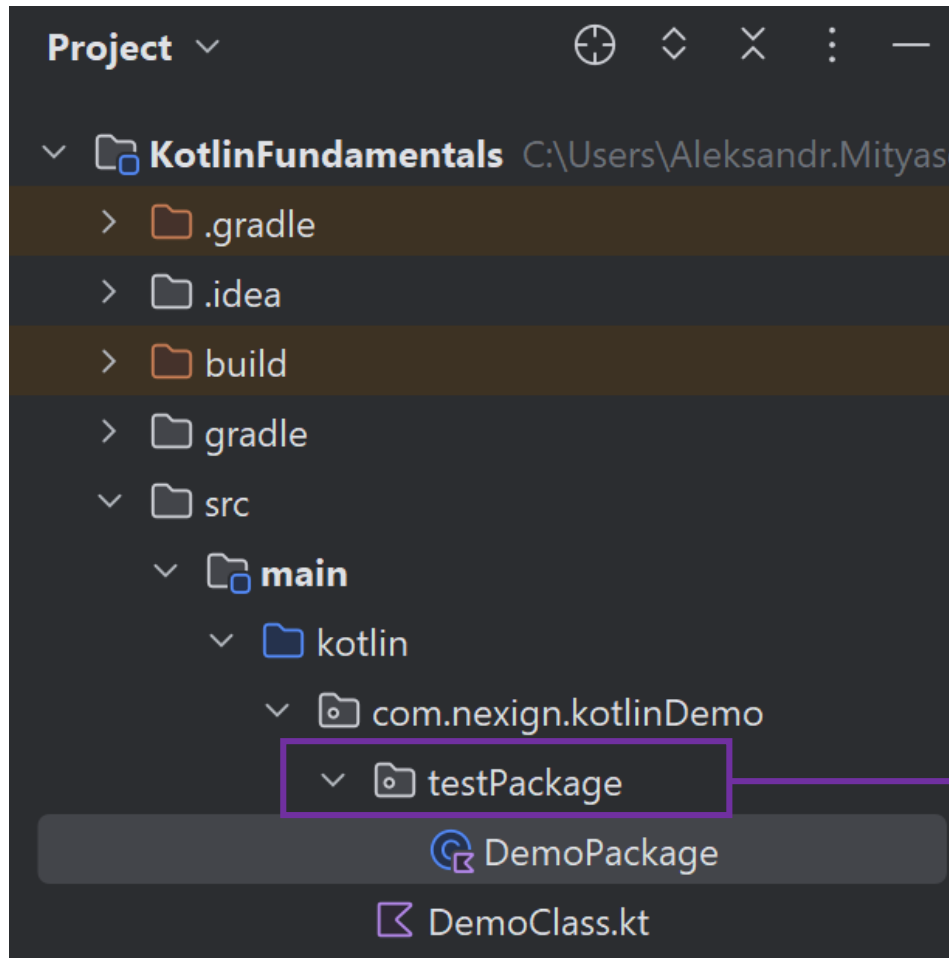
```
import kotlin.random.Random
```

```
class DemoClass(var a: Int) {  
    var b: String = "test string"  
    constructor(a: Int, b:String):this(a) {  
        println("$b and $a")  
    }  
}
```

```
fun someDemoFun(a: Int, b: Int): String {  
    return Random.nextInt(a,b).toString()  
}
```

```
fun main() {  
    val value =  
        DemoClass(2).someDemoFun(a = 3, b = 5)  
    println(value)  
}
```

# Пакеты



```
package com.nexign.kotlinDemo.testPackage  
  
class DemoPackage {  
}
```

# Импорты

```
import kotlin.random.Random
```

```
import kotlin.random.*
```

```
package com.nexign
```

```
fun someFun() {  
    println("SomeText")  
}
```

```
package com.nexign.newPack
```

```
import com.nexign.someFun as sf
```

```
fun main() {  
    sf()  
}
```

- ▶ Оператор **import** сообщает компилятору, где найти классы, на которые ссылается код
- ▶ Чтобы импортировать все содержимое пакета используйте «\*»
- ▶ Можно осуществлять импорт функций верхнего уровня
- ▶ Для разрешения коллизий имён, разрешено пользоваться псевдонимами:

```
import something as sth
```

# Импорты и пакеты

## Автоматический импорт:

```
kotlin.*  
kotlin.annotation.*  
kotlin.collections.*  
kotlin.comparisons.*  
kotlin.io.*  
kotlin.ranges.*  
kotlin.sequences.*  
kotlin.text.*
```

## Опционально, в зависимости от платформы:

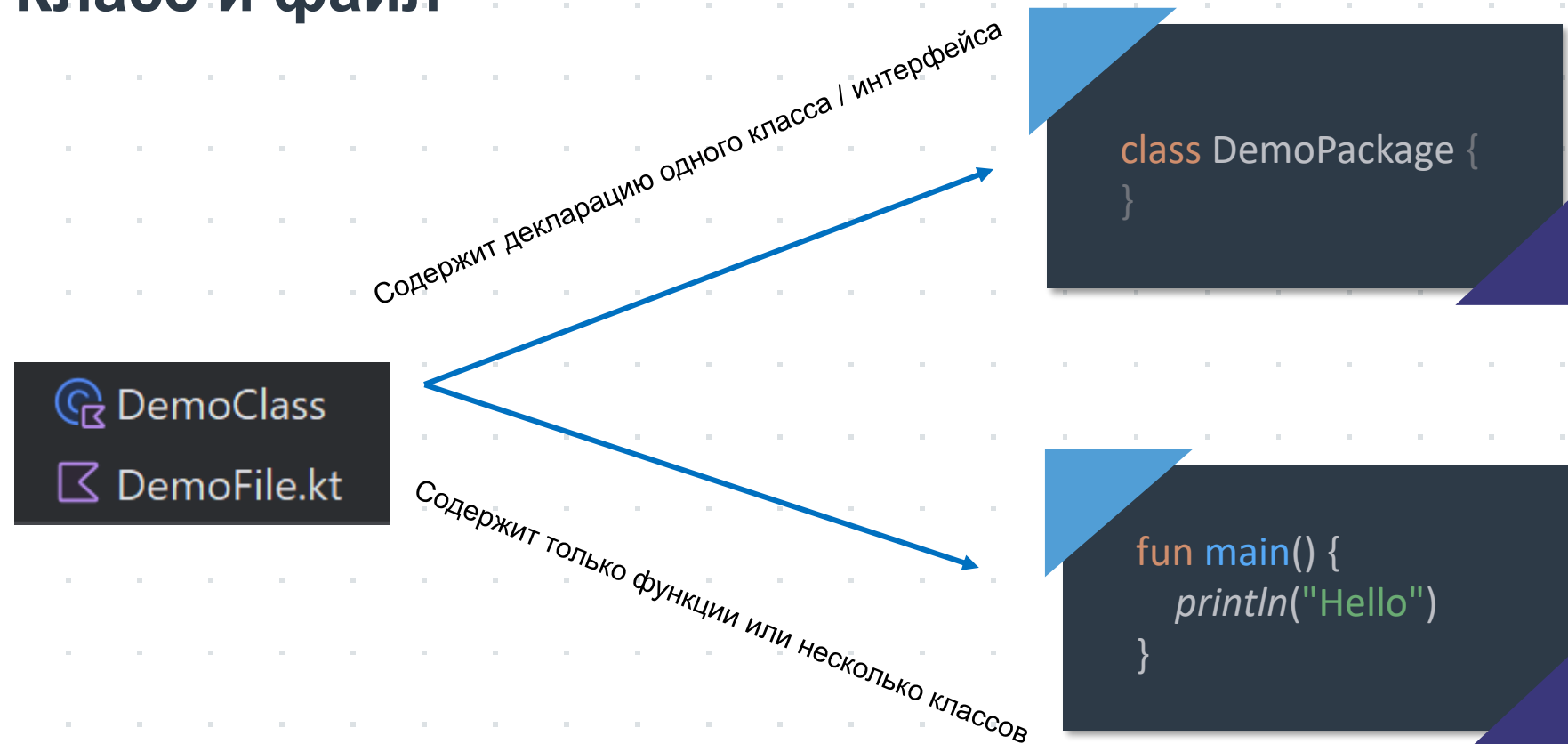
### VM:

```
java.lang.*  
kotlin.jvm.*
```

### JS:

```
kotlin.js.*
```

# Класс и файл



# Функция main

main() всегда должен находиться вне класса\объекта

```
fun main(args: Array<String>) {  
    println(args.getOrElse(0) { "Default" })  
}
```

fun main()



теперь свободен

- ▶ Точка входа в приложения
- ▶ Необходима для запуска
- ▶ В IntelliJ IDEA вызов через сниппет **psvma** или **maina**
- ▶ Параметр – строка передаваемая при запуске программ
  - ▶ может содержать доп. ключи (пример: *-d* запуск в дебаг. режиме)
- ▶ Может не иметь параметра
  - ▶ сниппеты **psvm** или **main**

# Комментарии

- ▶ Однострочный `// ...`
- ▶ Многострочный `/* ... */`
- ▶ Многострочный комментарий документации (javadoc) `/** ... */`

PACKAGE CLASS TREE DEPRECATED INDEX HELP

PREV CLASS NEXT CLASS FRAMES NO FRAMES

SUMMARY: NESTED | FIELD | CONSTR | METHOD DETAIL: FIELD | CONSTR | METHOD

dustin.examples.inheritance

**Class Animal**

java.lang.Object  
dustin.examples.inheritance.Animal

Direct Known Subclasses:  
Mammal

---

public abstract class Animal  
extends java.lang.Object

Animal.

**Constructor Summary**

Constructors

Constructor and Description

Animal()

---

**Method Summary**

| All Methods       | Instance Methods | Abstract Methods       | Concrete Methods      |
|-------------------|------------------|------------------------|-----------------------|
| Modifier and Type |                  | Method and Description |                       |
| void              |                  | breathe()              | Breathe.              |
| abstract void     |                  | verballyCommunicate()  | Communicate verbally. |

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

JavaDoc File

# Именованние объектов

▶ Классы



▶ Переменные



▶ Константы



▶ Функции



С большой буквы

Стиль CamelCase

*Пример: MyFirstClass*

С маленькой буквы

Использовать понятные по смыслу значения

*Пример: userId, todayTemperature*

Заглавные буквы. Разделители «\_»

*Пример: MAX\_VALUE*

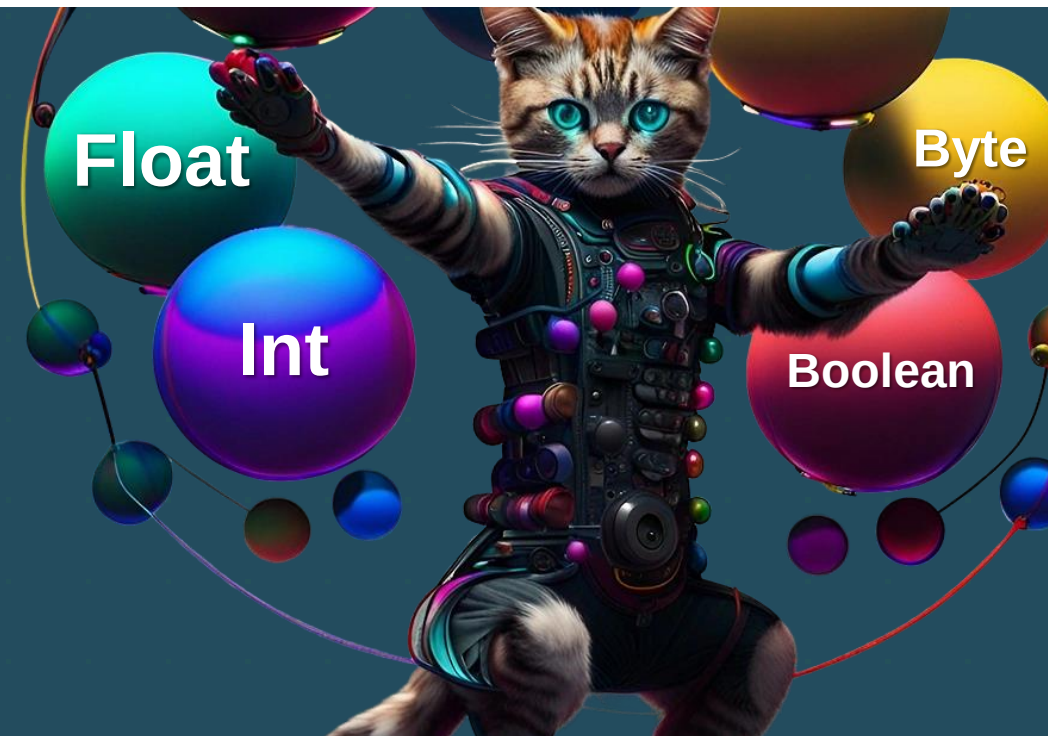
С маленькой буквы

Использовать глагольные выражения

*Пример: getCount(), setTemperature()*

# Типы данных и переменные

1. Прimitives и ссылочные типы
2. Типы данных в Kotlin
3. Инициализация переменных
4. Особенности числовых типов
5. Строки
6. Приведение типов
7. Операторы



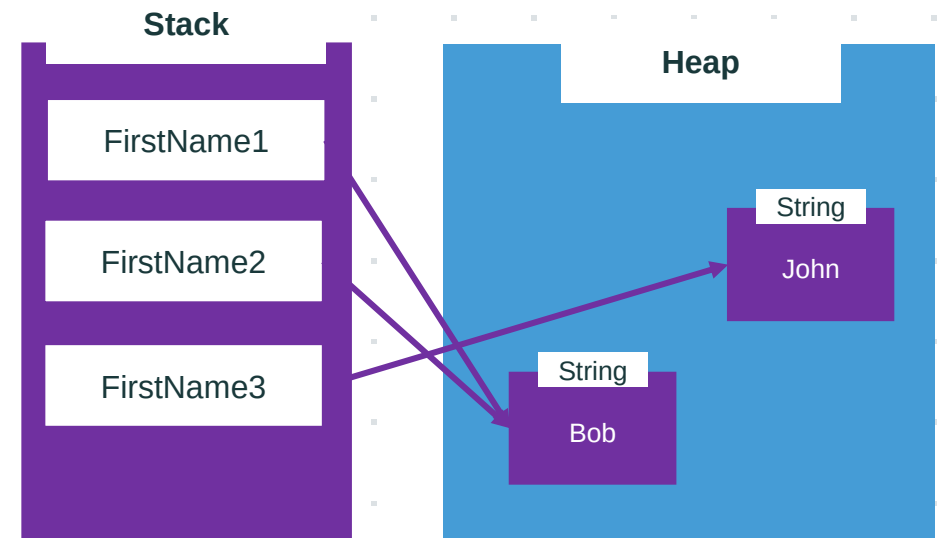
# Примитивные и ссылочные типы

## Примитивы



```
int a = 113;  
int b = a; // копирование по значению
```

## Ссылочные типы



# Базовые типы данных

| Тип     | Длина в байтах | Длина в битах | Диапазон                                                                |
|---------|----------------|---------------|-------------------------------------------------------------------------|
| Byte    | 1              | 8             | $[-128, 127]$ или $[-2^7, 2^7-1]$                                       |
| Short   | 2              | 16            | $[-32768, 32767]$ или $[-2^{15}, 2^{15}-1]$                             |
| Int     | 4              | 32            | $[-2147483648, 2147483647]$ или $[-2^{31}, 2^{31}-1]$                   |
| Long    | 8              | 64            | $[-9223372036854775808, 9223372036854775807]$ или $[-2^{63}, 2^{63}-1]$ |
| Float   | 4              | 32            | $[1.4e^{-45}, 3.4028235e^{38}]$                                         |
| Double  | 8              | 64            | $[4.9e^{-324}, 1.7976931348623157^{308}]$                               |
| Char    | 2              | 16            | $['\u0000', '\uffff']$ или $[0, 65535]$                                 |
| Boolean | 1 или 4        | 8 или 32      | true, false                                                             |

# Типы данных в Kotlin

- ▶ **Примитивных типов как в Java нет!**
  - ▶ ...а те что есть из Java используют autoboxing;
- ▶ У ссылочных типов есть свойства и функции члены;
- ▶ Хранятся в `kotlin.Primitives`
- ▶ У всех типов есть “зеркальный брат” – nullable-тип со знаком “?”



Как нет?

# Переменные

- ▶ **val** (от value) – неизменяемая ссылка. Переменной, объявленной с ключевым словом **val**, нельзя присвоить значение после инициализации. Такие переменные соответствуют финальным переменным в Java.
- ▶ **var** (от variable) – изменяемая ссылка. Значение такой переменной можно изменить. Такое объявление соответствует обычной (не финальной) переменной в Java.

```
var firstName: String = "Вася"  
firstName = "Саша"
```

```
val lastName: String = "Смирнов"  
//ошибка  
lastName = "Иванов"
```

# Type inference

Вывод типа переменной, на основе значения в правой части выражения

- ▶ Менять тип после инициализации запрещено
- ▶ Если значение не инициализировано, тип нужно указывать явно

```
val a: Int = 42
```

или

```
val a = 42
```

```
var a: Int = 42
```

```
//error
```

```
a = "42"
```

```
var a: Int
```

# Типы литералов

- ▶ Литералы это примитивные значения (строки, числа, символы, **true/false**).
- ▶ Единственный литерал, связанный с объектом это **null**
- ▶ Восьмеричные литералы не поддерживаются
- ▶ Нет преобразования между **boolean** и другими примитивными типами
- ▶ Базовый тип для целочисленных литералов – **Int\***
- ▶ Для чисел с плавающей запятой - **Double**

```
var a = 0b101101 // двоичная
var c = 45 // десятичная
var d = 0x2D // шестнадцатеричная
var pi = 3.14159
var piExp = 31415.9E-4
var x = 123456789
var num = 8_343_398.0
var stringHello1 = "\"Hello\"" // "Hello"
var stringHello2 = "Hello" // Hello
var ampersand1 = '&'
var ampersand2 = '\u0026' // шестнадцатеричная
var flag = false // или true
var emoji = "\ud83d\udc31" // cat 🐱
```

# Числовые типы. Целочисленные

- ▶ Нет неявного преобразования типов
- ▶ JVM хранит значения в виде примитивов
- ▶ Если нужна ссылка на **null**, используйте nullable-тип (**Int?**)
  - ▶ JVM обернёт его в Java-обёртку (**Integer**)
- ▶ Есть беззнаковые типы (**uByte**, **uShort**, **uInt**, **uLong**)
  - ▶ Приведение беззнакового типа к знаковому (и наоборот) - недопустимо

```
val x = 45L
val one = 1 // Int
val threeBillion = 3000000000 // Long
val oneLong = 1L // Long
val oneByte: Byte = 1
val two = 2
val twoByte: Byte = two //ошибка компиляции
```

# Приведение

- ▶ Оператор проверки типов `is`
- ▶ Оператор приведения «опасный» и «безопасный» (`as/ as?`)
- ▶ Методы приведения к примитивным типам\*
  - ▶ `toLong()`
  - ▶ `toString()`
  - ▶ `toInt()`...

```
var x: Long
var y: Int = 1000
x = y.toLong()
println(x::class)

var z: Number = 100
y = z as Int
println(y::class)
```

**ClassCastException**

# Числовые типы. С плавающей запятой

| Тип    | Размер (биты) | Значимые биты | Биты экспоненты | Разряды |
|--------|---------------|---------------|-----------------|---------|
| Float  | 32            | 24            | 8               | 6-7     |
| Double | 64            | 53            | 11              | 15-16   |

- ▶ Тоже нет неявного преобразования между **Float** и **Double**
- ▶ Чтобы явно указать тип **Float**, добавьте после значения f или F. Если такое значение содержит более 6-7 разрядов, оно будет округлено

# Строки

- ▶ Для работы с текстовыми данными в Java чаще всего используется класс **String**.
- ▶ Класс **String** имеет две фундаментальные особенности:
  - ▶ **immutable** (неизменный) класс;
  - ▶ **final** класс
- ▶ Класс **String** поддерживается на уровне виртуальной машины.
- ▶ Перегружен оператор "+".
- ▶ Взамен конкатенации рекомендуется использовать **String interpolation\***.

# String interpolation

Для вывода значений переменных или функций в строку можно использовать знак "\$" для интерполяции

```
val name = "Иван"  
val lastName = "Иванов"  
println("$name $lastName")  
println("${getShortName(name, lastName = lastName)}")
```



# Длинные строки

Для использования символа “\$” в строках требуется экранирование “\”

```
val kotlin = """
```

```
    Kotlin
```

```
    (@)
```

```
    (@)(@)
```

```
    (@)(@)
```

```
    """
```

```
println(kotlin)
```

```
    Kotlin
```

```
    (@)    (@)(@)    (@)(@)
```

# Null safety types

- ▶ Любой тип изначально не подразумевает включения null и запрещает такое присваивание на этапе компиляции.
- ▶ Чтобы допустить наличие null в типе, необходимо указать его со знаком “?”

```
//ошибка компиляции
```

```
val a:Int = null
```

```
//ok
```

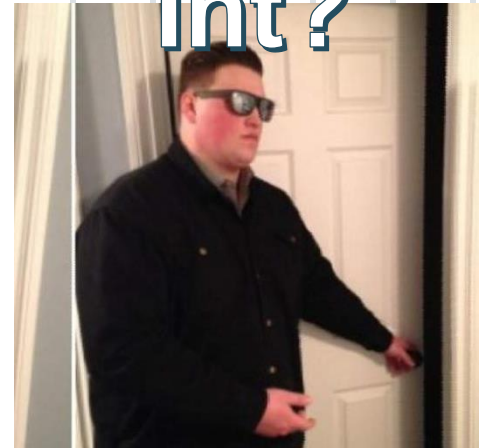
```
val b:Int? = null
```

null →

Int



Int?



# Операторы

|                       |                                                                   |
|-----------------------|-------------------------------------------------------------------|
| +, -, *, /, %         | математические операторы                                          |
| *                     | распаковка массива в параметр vararg                              |
| =                     | оператор присваивания                                             |
| +=, -=, *=,<br>/=, %= | расширенные операторы присваивания                                |
| ++, --                | операторы инкремента и декремента                                 |
| &&,   , !             | логические операторы 'и', 'или', 'не'                             |
| ==, !=                | операторы равенства (equals())                                    |
| ===, !==              | операторы ссылочного равенства                                    |
| <, >, <=, >=          | операторы сравнения (compareTo())                                 |
| [, ]                  | оператор индексированного доступа<br>(переопределенные get и set) |
| !!                    | утверждает, что выражение не null                                 |
| ?.                    | выполняет безопасный вызов                                        |

|    |                                                              |
|----|--------------------------------------------------------------|
| ?: | Элвис оператор                                               |
| :: | создает ссылку на элемент или ссылку на класс                |
| .. | создает диапазон                                             |
| :  | отделяет имя от типа в объявлениях                           |
| ?  | отмечает тип с нулевым значением                             |
| -> | разделяет параметры и тело лямбда-выражения                  |
| @  | вводит аннотацию                                             |
| ;  | разделяет несколько операторов на одной строке               |
| \$ | ссылается на переменную или выражение в<br>строковом шаблоне |
| _  | анонимный параметр                                           |

**Отличие от Java: Есть перегрузка операторов!**

# Practice 02 | Data types

1. val & var
2. equals, is, in and etc
3. null safety
4. string interpolation
5. type cast



# Промежуточный итог

- ❑ при объявлении переменной обязательно указывается степень ее изменяемости (val & var);
- ❑ Синтаксис описания типа **имя\_переменной: Тип**;
  - ❑ При чем если сразу инициализировать, то тип можно не указывать, т.к. он будет выведен из правой части;
- ❑ Все типы ссылочные;
- ❑ У любого ( в том числе и кастомного) типа есть nullable версия;
- ❑ Удобная работа со строками;
- ❑ Все операторы – это методы => могут быть перегружены.

# Функции I

1. Модификаторы видимости
2. Шаблон описания функции
3. Именованные аргументы
4. Smart casts

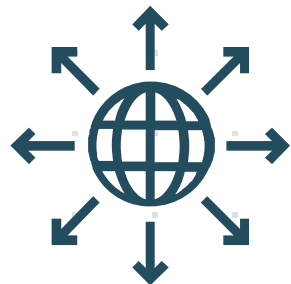


# Модификаторы видимости



**private**

доступ только из  
этого файла



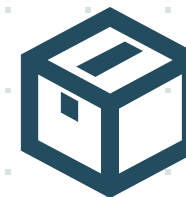
**public**

(по умолчанию)  
доступ всем



**protected**

доступен в  
текущем классе  
и в наследниках



**internal**

доступ всем  
внутри модуля\*

# Функции

Шаблон:

```
private open fun printFirstName(args: Array<String>): Unit {  
    println(args[0])  
}
```

- ▶ Если функция ничего не возвращает, то тип Unit (= void в Java) указывать не обязательно
- ▶ Если в теле функции одно действие, то фигурные скобки можно заменить знаком “=“

# Именованные аргументы

При вызове функции можно указывать имя параметра, если не помните на каком месте он стоит

```
fun getShortName(firstName: String, secondName: String = "Александрович", lastName: String) =  
    "${firstName[0]}.${secondName[0]}. $lastName"
```

.....

```
val shortName = getShortName("Иван", secondName = "Иванович", "Иванов")
```

- ▶ Параметр может иметь значение по умолчанию. В этом случае, его указание в списке аргументов не обязательно
- ▶ Именованные аргументы можно смешивать с позиционными только если последние стоят на “своих местах”. Сами по себе именованные аргументы можно перечислять в любом порядке

# Smart casts

```
fun shorten(name: Any): String {  
    if (name is String) {  
        return name[0] + "."  
    }  
    return ""  
}
```

- ▶ **Any = Object – верхушка иерархии классов**
- ▶ При выполненной проверке типов, name автоматически приводится к типу String
- ▶ Смарткасты работают только тогда, когда компилятор может гарантировать, что переменная не изменится между проверкой и использованием

# Управление потоком

1. Conditions (if/elvis/when)
2. Null safety
3. Циклы
4. try/catch/finally



# If/Elvis

- ▶ Условный оператор работает как обычно

```
fun getFirstName(names: Array<String>): String? {  
    if (args.size > 0) return args[0]  
    else return null  
}
```

- ▶ Но может выступать в качестве выражения (замена ?: в Java)

```
fun getFirstName(names: Array<String>): String? {  
    return if (args.size > 0) args[0]  
    else null  
}
```

```
fun getFirstName(names: Array<String>) =  
    if (args.size > 0) args[0]  
    else null
```

# If/Elvis

**x ? : y**

Если X не null, то выполнить X, иначе выполнить Y

```
val john : Person? = Person("John", 32)  
  
val age = john?.age  
val offsetAge = age + 1 /* compile error */
```

```
val age = john?.age ?: 25  
val offsetAge = age + 1 /* no compile error */
```



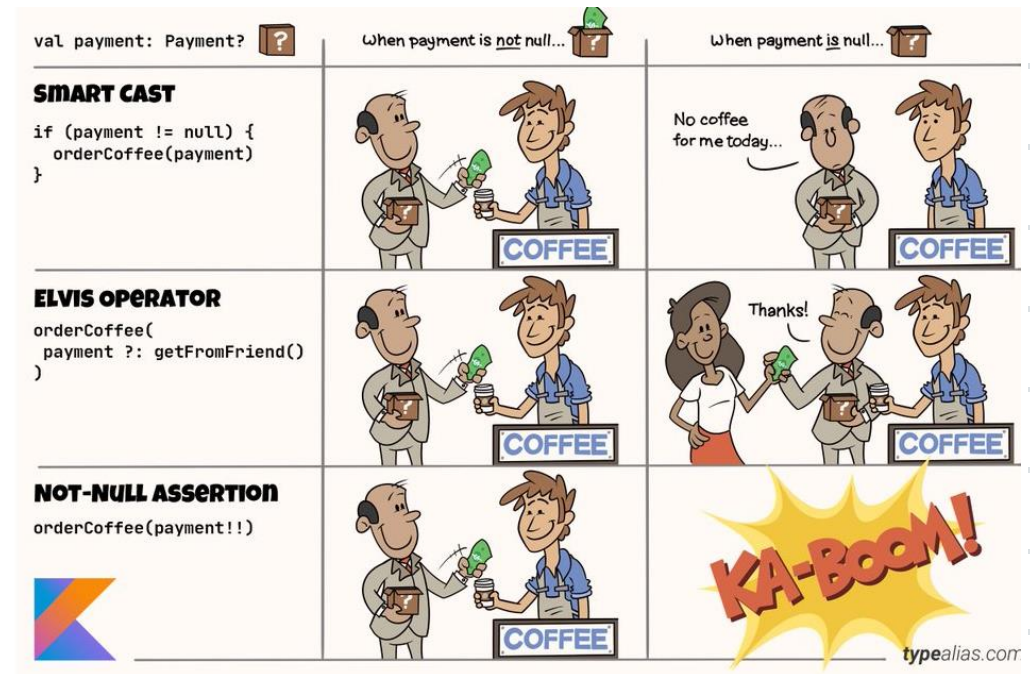
# Null safety

Операции с nullable типами по умолчанию запрещены

```
val b:String? = null
//ошибка вызова
println(b.length)
```

Для их активации можно использовать:

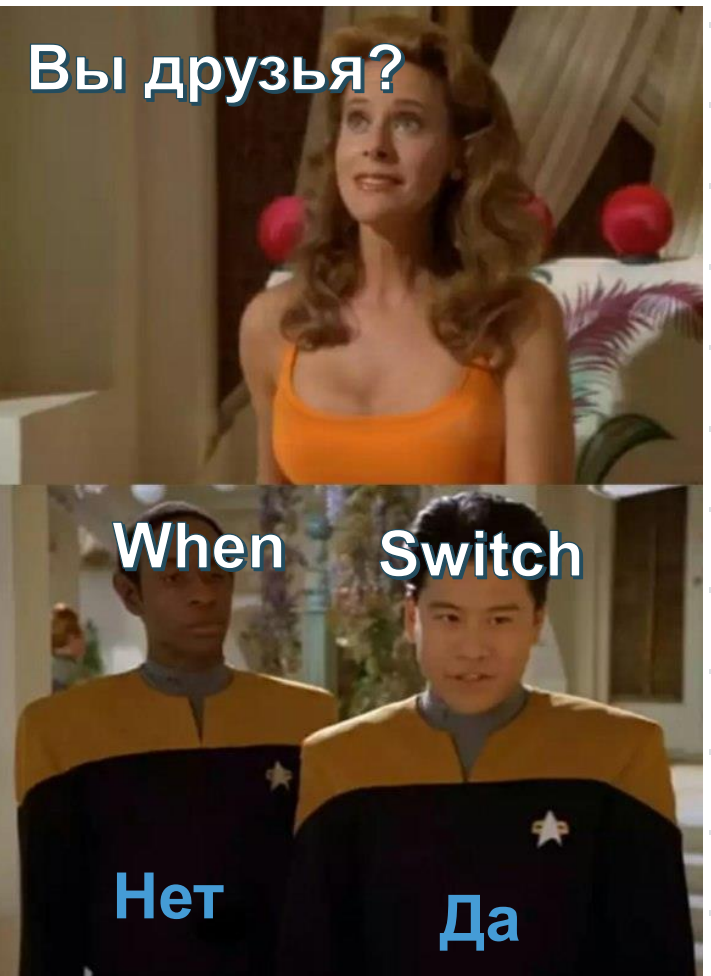
- ▶ Явную проверку (if x != null) – срабатывает smart cast
- ▶ Elvis оператор (x?: 0) – если null, то вернёт то что после “:?”
- ▶ Оператор-бабах (x!!.length) – рекомендуется избегать
- ▶ Безопасный вызов (x?.length) – если x null, то вернёт null



# When

- ▶ Switch на стероидах
  - ▶ ...или switch-expression (java 14+)
- ▶ **break** не нужен
- ▶ **when** – это выражение

```
val pointType = org.apache.commons.math3.geometry.euclidean2d.PointType
val value = org.apache.commons.math3.geometry.euclidean2d.PointType.BELARUS as B
when(value) {
    10 -> println("в диапазоне")
}
fun main() {
    println("большее 5") //не работает
    val LongInt = R> "Number"
    val DoubleFloat = R> "country"
    else R> "Ужасный number"
}
    B -> "белорусский"
    else -> "другое"
//else нужен, если не перечислили всё в Enum
```



# Циклы

## ▶ do-while/ while – как в Java

```
var num = 100
do {
    println("Loop: $num")
    num++
}
while (num<=105)

num = 10
while(num>=5){
    println("Loop: $num")
    num--
}
```

## ▶ for – много удобных штук

```
val oneToTen = 1..10 // [1,10]
println(oneToTen)
for (i in oneToTen) println(i)
//for (i in 1..10) println(i)
//for (i in 1 until someArray.size)

val array = arrayOf("a", "b", "c")
for (i in array.indices) {
    println(array[i])
}
for ((index, value) in array.withIndex()) {
    println("the element at $index is $value")
}
```

Kotlin поддерживает привычные операторы break и continue в циклах

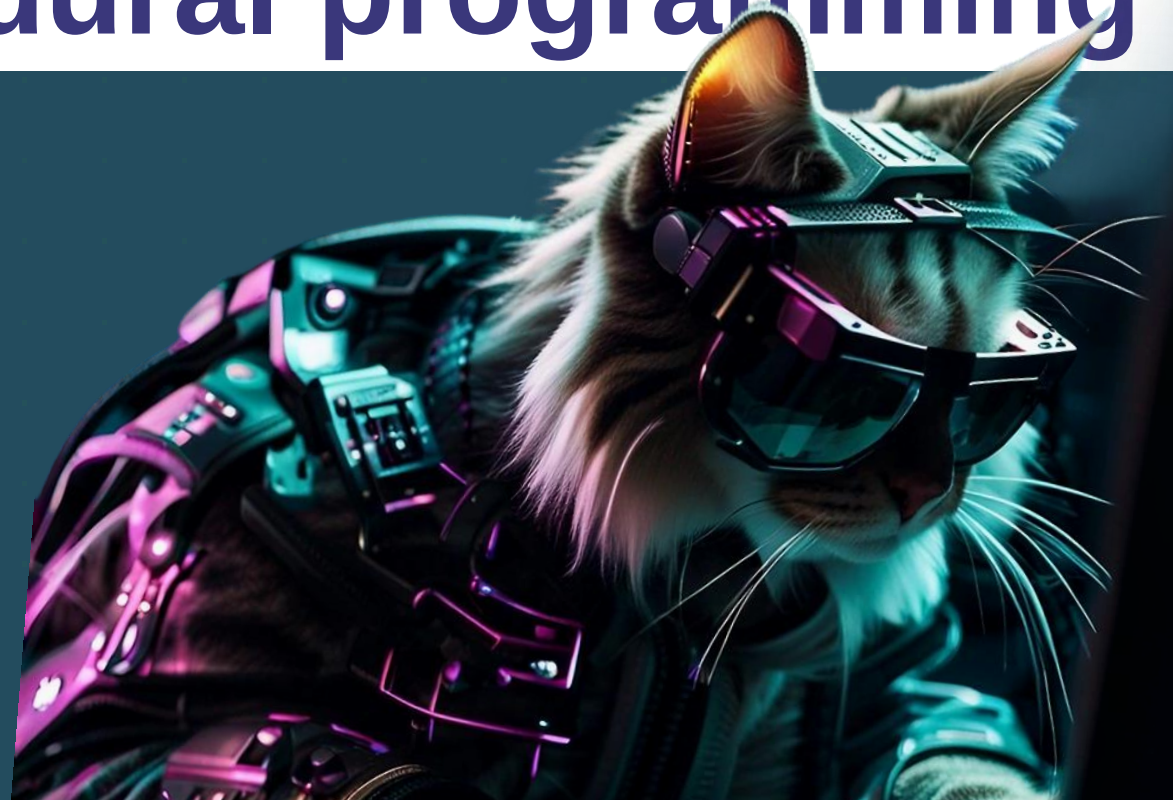
# try/catch/finally

- ▶ **try** – это выражение
  - ▶ его тип (как и тип return) - **Nothing**
- ▶ нет **checked** исключений => не нужно указывать **throws** в сигнатуре (можно ставить аннотацию **@Throws**)

```
var numExample: String = "0.0."  
val toIntExample = try {  
    Integer.parseInt(numExample)  
} catch (ex: NumberFormatException) {  
    null  
}  
println(toIntExample) //null
```

# Practice 03 | Procedural programming

1. functions
2. smart casts
3. conditions
4. loops
5. null safety



# Массивы

1. Массивы
2. Операции с массивами
3. Диапазоны
4. Enums



# Способы создания массива

С помощью функций:

```
val arr: Array<Int> = arrayOf(1,2,3)
val nullArr = arrayOfNulls<Int>(3)
val emptyArr = emptyArray<Int>()

//java способ вывода массива
println(Arrays.toString(arr))

//kotlin способ
println(nullArr.contentToString())
```

С помощью конструктора:

```
val constArr = Array(3, {0})
```

Количество элементов — 3  
Лямбда выражение — {0}

```
//двумерный массив 3*3
val squareArr = Array(3) {Array(3) {i -> i+1}}

//вывод в глубину
println(squareArr.contentDeepToString())
```

Массивы в Котлине представлены классом `Array<T>`

# Ещё массивы

- ▶ Есть массивы типов: IntArray, ByteArray, ShortArray и т.д.
  - ▶ Не наследуются от Array, но обладают тем же функционалом
  - ▶ Каждый имеет соответствующую фабричную функцию (intArrayOf(), shortArrayOf())
- ▶ Методы get(), set() переопределены – [ ]
- ▶ **Массивы неизменяемы**

```
var intArr: IntArray = intArrayOf(1,2,3)
```

```
println(intArr.get(0))
```

```
// ==
```

```
println(intArr[0])
```

```
intArr = intArrayOf(3,4,5) //был создан новый массив
```

```
val boolArr = booleanArrayOf(true, true)
```

```
boolArr[0] = false //ОК, хоть и val
```

```
intArr.size //длина массива
```

```
intArr.average() //среднее значение
```

```
intArr.sum() //сумма
```

```
intArr.forEach { x-> println(x+1) }
```

# Диапазоны

- ▶ Могут быть не только числовые
  - ▶ для этого итерируемый тип должен содержать свойства **first**, **last**, **step** и реализовывать интерфейс **Progression<T>**
- ▶ Стирается на этапе компиляции

```
for (c in 0..9)  
    print (c)
```

```
for (c = 0; c <= 9; c++)  
    print(c)
```

```
val a = 1..10 // include 10  
val b = 1 until 10 //not include  
val c = 5 downTo 1 //reverse order  
val d = 1..10 step 2  
d.forEach { println(it) } //1,3,5,7,9  
// check if d contains 13  
val e = if (13 !in d) {d + 13} else { d }
```

# Enum

- ▶ Список именованных констант
- ▶ Обычные классы: имеют поля, методы, могут реализовывать интерфейсы, но не могут наследоваться
  - ▶ `values()` – список значение
  - ▶ `valueOf()` – значение по имени
- ▶ просто Enum из java

```
val enum = 1
enum class Country(val capital: String, val currency: String) {
    RUSSIA("Москва", "рубль"),
    BELARUS("Минск", "беларусский рубль"),
    JAPAN("Токио", "йена"),
    WAKANDA("Birnin Zana", "Wakandan Dollar");
    fun info() =
        println("Информация о стране: \n$name\n$capital\n$currency")
}

fun main() {
    val country = Country.RUSSIA
    country.info()
}
```

# Practice 04 | Arrays

1. array initialization
2. iterate over arrays/ranges
3. explore some functions



# Итог

- ❑ Kotlin решает многие проблемы старых версий Java
- ❑ null явно учитывается в системе типов
- ❑ Есть функции
- ❑ Есть вывод типов и string interpolation
- ❑ Много сахара



# Q&A



# Спасибо!



Aleksandr.Mityasov



@mityasov



Aleksandr.Mityasov@nexign.com

В настоящем документе содержится не предназначенная для публикации конфиденциальная информация и материалы, принадлежащие АО «Нэксайн» или его аффилированным лицам, охраняемые российским и международным законодательством об авторском, патентном праве, праве на секрет производства (ноу-хау), о товарных знаках и иных средствах индивидуализации.

Программное обеспечение АО «Нэксайн» и связанная с ним документация могут распространяться и использоваться только на основании отдельного лицензионного соглашения с АО «Нэксайн».

Любое использование, копирование, воспроизведение, публикация, обнародование, передача, изменение, переработка или перевод настоящего документа и содержащихся в нем информации и материалов, полностью или частично, в любой форме и любыми средствами без предварительного письменного разрешения АО «Нэксайн» строго запрещено.

Если настоящий документ предоставляется с программным обеспечением, лицензированным АО «Нэксайн», информация в нем предоставляется в соответствии с условиями гарантии, предоставляемой с лицензией на программное обеспечение.

Если настоящий документ не предоставляется с лицензионным программным обеспечением, информация в нем предоставляется «как есть» без каких-либо гарантий.

В обоих случаях АО «Нэксайн» не предоставляет никаких иных гарантий в отношении настоящего документа и содержащихся в нем информации и материалов, явных или подразумеваемых, включая, помимо прочего, подразумеваемые гарантии в отношении их качества, коммерческой пригодности, пригодности для определенной цели, ненарушения прав третьих лиц.

АО «Нэксайн» или его аффилированные лица ни при каких обстоятельствах не несут ответственность за любые прямые или косвенные убытки, случайные расходы или штрафы, возникшие в результате использования или невозможности использования материалов и/или информации, содержащихся в настоящем документе.

АО «Нэксайн» принимает надлежащие меры для обеспечения качества информации и материалов, содержащихся в настоящем документе. Однако документ может содержать технические неточности или опечатки, отсутствие ошибок не гарантируется.

Настоящий документ может быть изменен или дополнен без какого-либо уведомления. АО «Нэксайн» не обязуется обновлять настоящий документ на основе изменений в продуктах или услугах, как собственных, так и третьих лиц. АО «Нэксайн» может вносить улучшения или изменения в продукты или услуги, описанные в настоящем документе, в любое время без предварительного уведомления.



# Класно и обЪЕКТИВНО: ООП в Kotlin



# План урока

- ▶ Коротко об ООП
- ▶ Структура класса
- ▶ Наследование и прочие родственные связи
- ▶ Шаблоны проектирования и Kotlin

# Давным-давно...

- ▶ Функциональная программа отвечает на вопрос “что?”
- ▶ Императивная программа отвечает на вопрос “как?”

Кто-то попытался добавить ФП в свой императивный мир

- ▶ Java
- ▶ C++
- ▶ C#

Кто-то попытался добавить ИП в свой функциональный мир

- ▶ OCaml
- ▶ Clojure

Кто-то попытался объединить две стороны

- ▶ Scala
- ▶ Kotlin



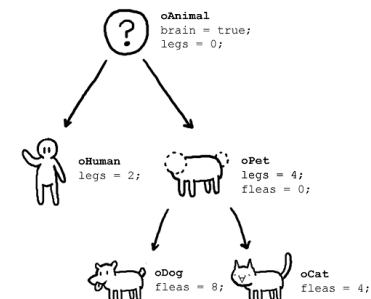
# Что за ООП?

- ▶ Все — это объект
- ▶ Объекты содержат данные (поля и/или свойства)
- ▶ Объекты могут обмениваться сообщениями (вызовы методов)

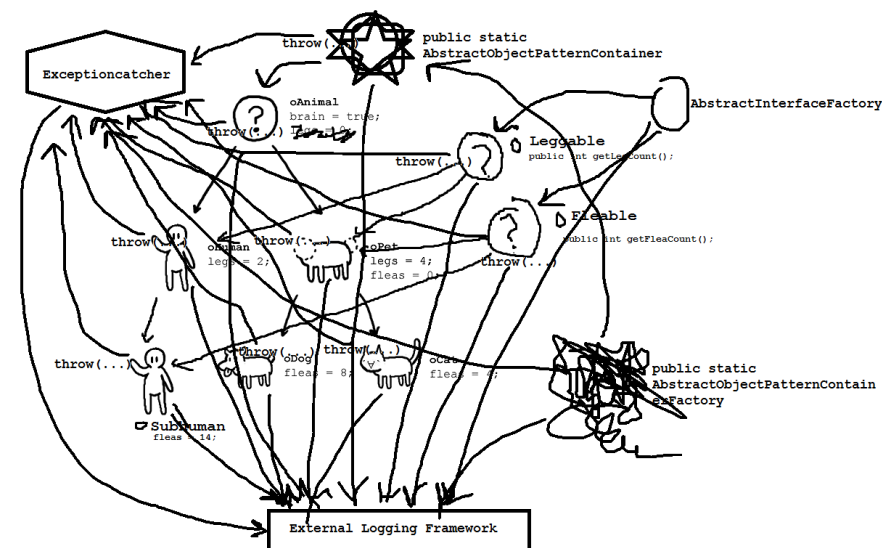
## И, конечно:

- ▶ Наследование
- ▶ Полиморфизм
- ▶ Инкапсуляция

### ООП, которое пользователи заявляют



## Что на самом деле происходит



# ООП Java

- ▶ Объект – область фиксированного значения в куче, обладающая фиксированной структурой и содержащая заголовок и полезную информацию в виде набора значений
- ▶ Значение – ссылка может ссылаться на объекты или на null
- ▶ Новый объект не может быть равен (==) никакому ранее созданному объекту
- ▶ Любой код — метод некоторого класса
- ▶ Любые данные хранятся в полях некоторого класса
- ▶ Любые типы данных (исключая примитивные, но включая массивы) — наследники класса `java.lang.Object`

# ООП Kotlin

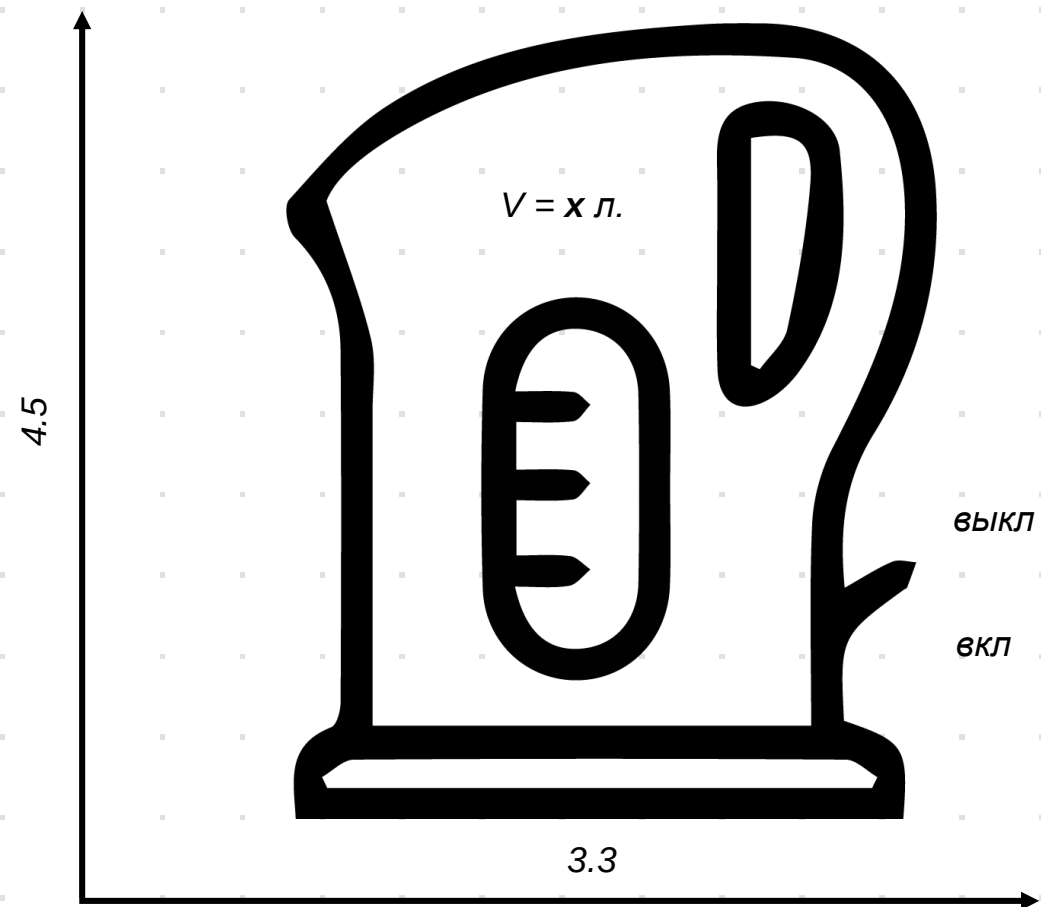
- Всё примерно то же самое!
- Ссылаться на null можно, но для получения, потребуется null-тип
- Бывают функции и классы
- Примитивов нет. А все данные – наследники от Any?

# План урока

- ▶ Коротко об ООП
- ▶ **Структура класса**
- ▶ Наследование и прочие родственные связи
- ▶ Шаблоны проектирования и Kotlin

# Класс

- ▶ Класс представляет собой абстрактное описание структуры реальных объектов
- ▶ Объекты создаются в оперативной памяти машины на основе такого описания, которое представляет собой перечисление свойств, имеющих у объектов и методов, которые объектам доступны
- ▶ Объект представляет собой совокупность конкретных значений свойств и методов, работающих с этими свойствами
- ▶ Вся логика работы java-приложений построена на взаимоотношении классов и объектов



# Структура класса

## ► Модификаторы

- видимости (по умолчанию public)
- расширяемости (по умолчанию final)
- другое\*

## ► Имя

- не обязательно совпадает с именем файла (в отличие от Java)

## ► Конструктор

## ► Поля

- не имеют значений по умолчанию и требуют инициализации
- **lateinit** – пообещать, что инициализируете позже

```
private open class Player {  
    val name: String = "Sir Arthur"  
    lateinit var sword: AbstractSword  
    // не сработает для базовых типов  
}
```

# Свойства

- ▶ Это поля
  - ▶ ...но не совсем
- ▶ Встроены **getter** и **setter**
  - ▶ ... но их можно и написать
  - ▶ **Теневые поля (backing field)** – нужны, чтобы избежать рекурсивный вызов свойства
- ▶ Упрощённая инициализация через конструктор
  - ▶ Можно задать значение по умолчанию

## Синтаксис объявления свойства

```
var <propertyName>[: <PropertyType>] [= <property_initializer>]  
    [<getter>]  
    [<setter>]
```

```
class Player constructor (name: String,  
    val playerClass: PlayerClasses?,  
    fun main() {  
        var xp: Int = 0  
        while (xp < 100) {  
            xp = (xp % (100 * modifier)).toInt() // xp - создаётся экземпляр, ему не нужен  
            val field = Player("Sir Arthur", 100)  
            modifier += 0.75  
            player.level // player.getLevel()  
            return field  
            player.level = 1 // player.setLevel(1)  
        }  
    }  
    set(value) {  
        xp = (value * modifier * 100).toInt()  
        field = value  
    }
```

# Конструкторы

**Конструктор** – набор необходимых параметров, необходимых для инициализации объекта

- ▶ В Kotlin есть 2 вида конструкторов:
  - ▶ Основной (*primary*)
  - ▶ Дополнительные (*secondary*)
- ▶ Ключевое слово **constructor** можно не указывать для основного конструктора, если нет модификаторов и аннотаций
- ▶ Основной конструктор не имеет встроенного блока инициализации
- ▶ Дополнительный конструктор обязательно должен ссылаться на основной **:this()**

```
class Player constructor (name: String, hp: Int) {  
    private val _name = name  
    lateinit var sword: AbstractSword  
    var hp: Int  
    ...  
  
    constructor(name: String): this(name, 100) {  
        sword = if (name.contains("Arthur"))  
            Excalibur()  
        else OrdinarySword()  
    }  
}
```

# Инициализация

## init block

Исполняемый участок кода, срабатывающий при инициализации объекта

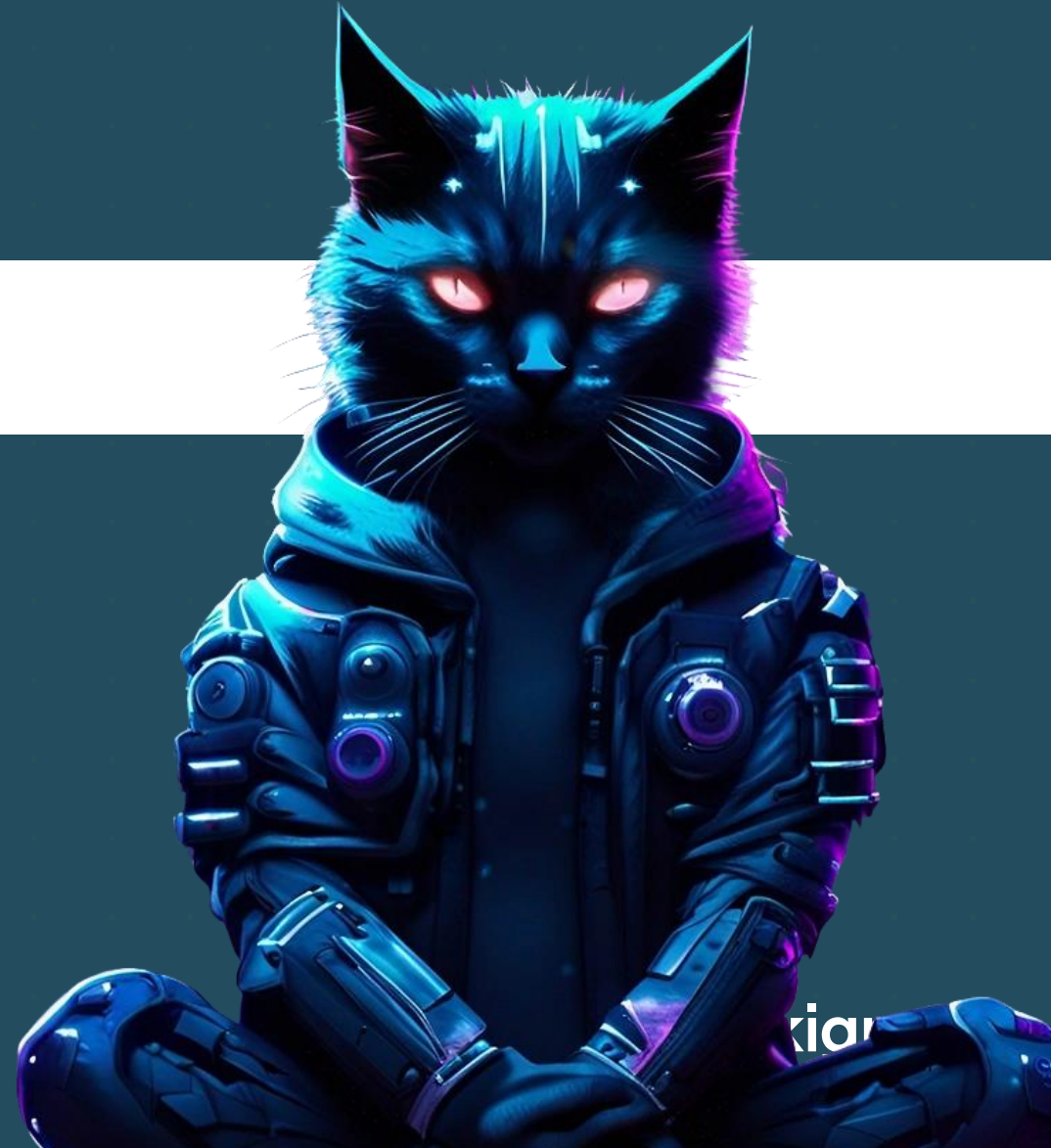
```
var hp: Int
init {
  this.hp = when (playerClass) {
    WARRIOR -> 200
    MAGE -> 50
    THIEF -> 70
    ARCHER -> 100
  }
  else -> hp
}
```

► Можно использовать для инициализации свойств

```
init {
  //java style
  this.name = name
  this.hp = hp
}
```

# Practice 01 | Classes

1. Create class
2. Create constructor
3. Initialize properties



# Псевдонимы типов

## typealias

Псевдонимы типов предоставляют альтернативные имена для существующих типов.

### Шаблон:

```
typealias NewName = LongOldName<Class>
```

```
//упрощаем обращения к коллекциям
typealias NodeSet = Set<Network.Node>
typealias FileTable<K> = MutableMap<K, MutableList<File>>
//псевдонимы функциональных типов
typealias MyHandler = (Int, String, Any) -> Unit
typealias Predicate<T> = (T) -> Boolean
//Именование вложенных классов
class A {
    inner class Inner
}
class B {
    inner class Inner
}
typealias AInner = A.Inner
typealias BInner = B.Inner
```

# Компаньоны

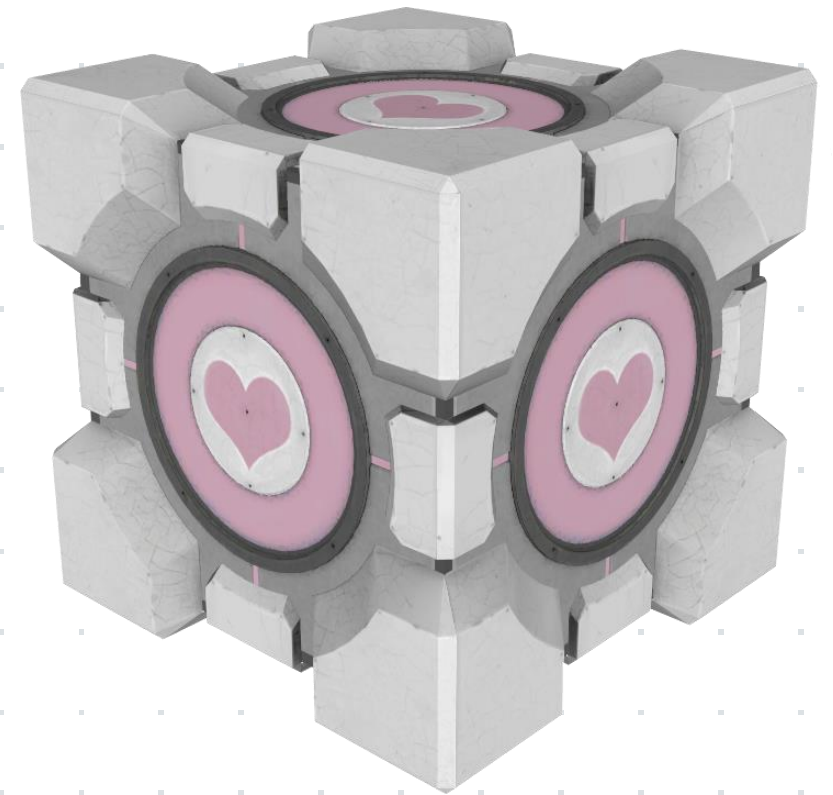
## Companion object

Замена **static**\* полей и методов из Java

- ▶ принадлежит классу
- ▶ только один компаньон на класс
- ▶ поля класса могут дотянуться до компаньона, но не наоборот
- ▶ может не иметь имени и отзываться на **“Companion”**

```
companion object Modifier {  
    var modifier = 1.1  
    fun hardcoreModeOn(): Boolean {  
        //нет доступа к полям класса - ошибка  
        hp-=50  
        modifier = 0.75  
        return true  
    }  
}
```

```
//вызов компаньона  
Player.Companion.modifier = 1.2  
Player.Modifier.hardcoreModeOn()  
Player.modifier = 1.2
```



# Методы

функции - члены

это алгоритмы, обрабатывающие данные(состояния)

... если проще – это функции внутри классов

- ▶ Могут быть перегружены (**overloading**)
  - ▶ одно имя, но разный список\порядок параметров
- ▶ Могут быть переопределены в наследнике (**overriding**)
  - ▶ и даже базовые операции можно! (+\ - \ \* \ % и т.д.)

```
fun changeWeapon(weapon: Weapon, hand: Hand) {  
    println("You are equip $weapon in $hand")  
}  
  
fun changeWeapon(weapon: Weapon) {  
    println("You are equip $weapon in two  
    hands")  
}  
  
override fun death() {  
    hp = 0  
    println("You are dead")  
}
```

# Вложенные классы

## inner classes

- ▶ Любой класс можно вложить в другой
- ▶ Внутренние классы, по умолчанию, не имеют ссылки на внешние
- ▶ Обращение к вложенным классам:  
`ИмяВнешнегоКласса.ИмяВложенногоКласса.метод()`
- ▶ Для доступа к внешнему состоянию нужно отметить вложенный класс как **inner**
- ▶ Если свойство или функция имеет то же имя что и у внешнего класса, то для обращения ему требуется конструкция:

`this@названиеКласса.имяСвойстваИлиФункции`

```
fun main() {  
    Inventory.Transaction().pay(1200);  
}  
class Inventory {  
    private var sum: Int = 3450  
    fun display() {  
        println("sum = $sum")  
    }  
}
```

```
class Transaction {  
    fun pay(s: Int) {  
        //нет доступа к  
        // внешнему состоянию  
        sum -= s  
        display()  
    }  
}
```

```
inner class Transaction {  
    var sum = 0  
    fun pay(s: Int) {  
        //изменение внешнего поля  
        this@Inventory.sum -= s  
        display()  
    }  
}
```

# План урока

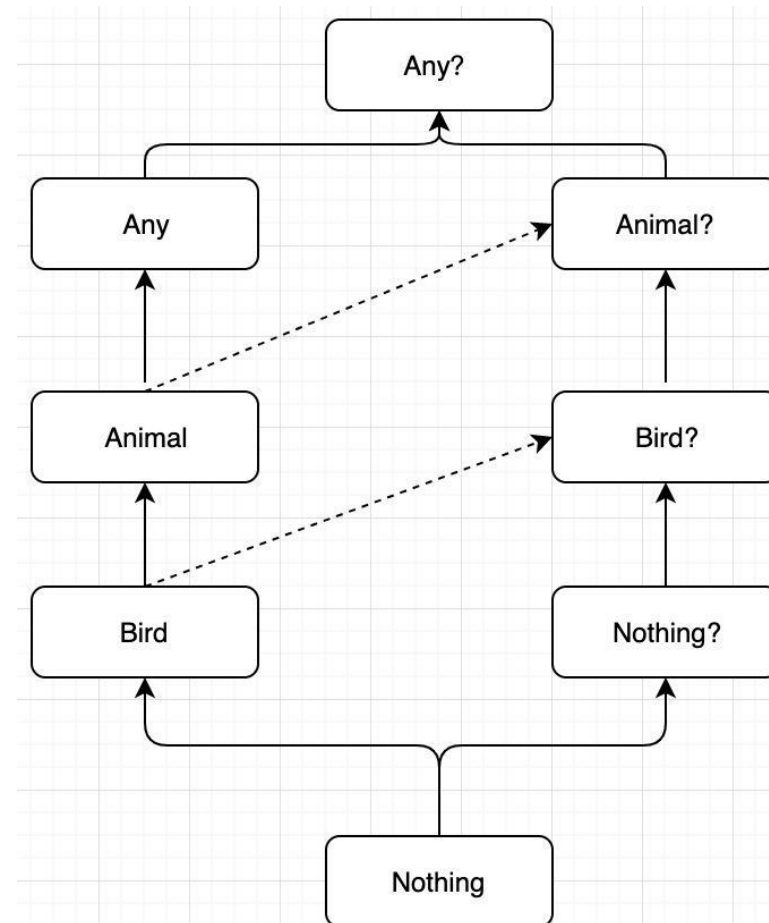
- ▶ Коротко об ООП
- ▶ Структура класса
- ▶ **Наследование и прочие родственные связи**
- ▶ Шаблоны проектирования и Kotlin



# Наследование

Механизм, позволяющий строить новые классы на основе первоначального

- ▶ Все наследники **Any/Any?**
- ▶ Каждый класс неявно связан со своим **nullable** - типом
- ▶ По умолчанию классы заблокированы для наследования (**final**)
- ▶ Чтобы сделать класс наследуемым, его нужно пометить ключевым словом **open**
- ▶ Нет множественного наследования

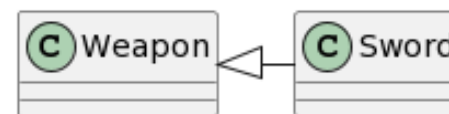


# Наследование

```
open class Weapon(val name: String) {  
    ...  
}
```

```
class OrdinarySword : Weapon("Sword") {  
    ...  
}
```

- ▶ Если у класса есть основной конструктор, базовый тип может (и должен) быть проинициализирован там же, с использованием параметров основного конструктора.
- ▶ Если у класса нет конструктора, то родитель всё равно будет вызываться каждый раз при инициализации объекта потомка, при помощи неявной **super()** ссылки



# Overriding

- ▶ Для переопределения поля или метода:
  - ▶ родитель должен иметь модификатор **open** на указанном члене класса
  - ▶ потомок должен иметь модификатор **override** на указанном члене класса
- ▶ Член класса помеченный **override** сам по себе **open** и может быть переопределён в производных классах
  - ▶ для запрета переопределения используется модификатор **final**
- ▶ Для вызова члена класса родителя используется конструкция **super.методИлиСвойство**

```
open class NPC(open val name: String = "John Doe",
               private var hp: Int = 50) {

    open fun walk() {
        println("$name is walking")
    }
}

open class Guard: NPC {
    //конструктор пустой, потому что есть значения по умолчанию
    constructor(): super()
    constructor(name: String): super(name)
    //ссылка на конструктор Guard с одним параметром
    constructor(name: String, hp: Int) :this(name)

    override val name: String
        get() = "sir ${super.name}"

    final override fun walk() {
        super.walk()
        println("$name patrol some area")
    }
}
```

# Абстрактный класс

- ▶ Класс, в котором хотя бы один из методов не реализован, должен быть помечен как **abstract**
  - ▶ каждый не реализованный метод, так же помечается как `abstract`
- ▶ Нереализованные методы в классе возникают двумя способами:
  - ▶ явно объявлены как **abstract**
  - ▶ унаследованы из абстрактных классов или интерфейсов и не переопределены.
- ▶ Абстрактные классы нельзя инстанцировать
- ▶ Абстрактный класс по умолчанию **open** и отдельно это указывать не нужно

# Абстрактный класс

```
abstract class NPC {  
    abstract val name: String  
    var hp = 50  
    //не реализованный  
    abstract fun walk()  
}  
//интерфейс тоже имеет не реализованный метод  
interface Mortal {  
    fun death()  
}  
//наследуется от абстрактного класса и реализует интерфейс  
abstract class Guard(name: String, hp: Int) : NPC(), Mortal {  
    //переопределяет поле и методы  
    override val name: String = "sir $name"  
    override fun walk() {  
        println("$name patrol some area")  
    }  
    abstract override fun death()  
}
```

sir



Abstract Guard

nexign

# Интерфейс

- ▶ Предназначен для выделения общего поведения у несвязанных классов (*Closable*, *Comparable*)
- ▶ Определяется ключевым словом **interface**
- ▶ Как и абстрактный класс, может не иметь реализации методов
  - ▶ при этом помечать такие методы как **abstract** не нужно
- ▶ Интерфейс не может хранить переменные экземпляров
  - ▶ но может реализовывать методы доступа
- ▶ Нельзя создавать экземпляры интерфейса
- ▶ Классы могут реализовывать множество интерфейсов



# Интерфейс

```
interface Destroyable {  
    //абстрактное свойство  
    val animation: Animation  
    //так нельзя  
    val piece: Int = 100  
    //а так можно  
    get() = 100  
    //абстрактный метод  
    fun destroy()  
    //метод по умолчанию  
    fun animate() {  
        animation.run()  
    }  
}
```

```
class Box : Destroyable {  
    override val animation: Animation  
        get() = Explode(piece)  
    override fun destroy() {  
        animate()  
        this.setVisible(false)  
        createMesh(DestroyBox())  
    }  
}
```

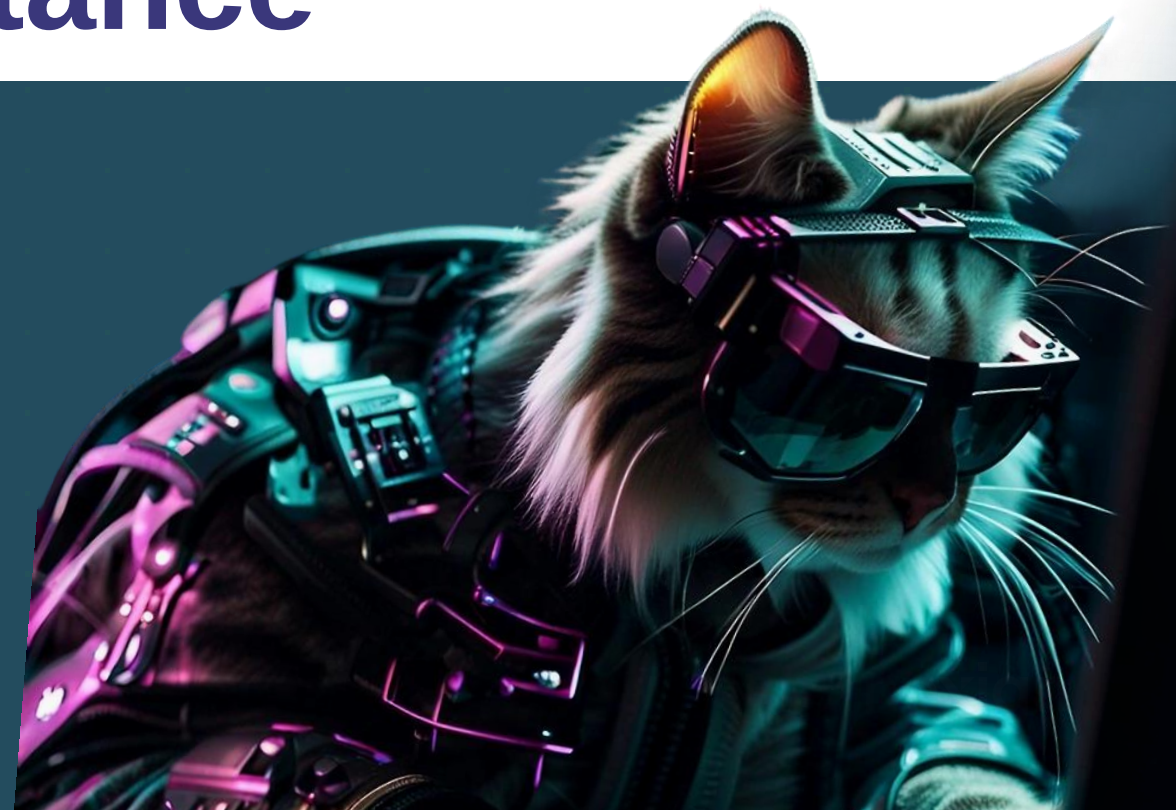
# Устранение противоречий

```
interface Destroyable {  
    fun destroy() {  
        setVisible(false)  
    }  
}  
  
interface Mortal {  
    fun destroy() {  
        hp = 0  
    }  
}  
  
class AliveMechanism: Destroyable, Mortal {  
    override fun destroy() {  
        super<Destroyable>.destroy()  
        super<Mortal>.destroy()  
    }  
}
```

- ▶ При реализации нескольких интерфейсов с одним и тем же именем метода, то для указания конкретной функции указывается имя класса/интерфейса в угловых скобках.

# Practice 02 | Inheritance

1. Class hierarchy
2. Abstract classes
3. Interfaces



# Запечатанные классы

## Sealed classes

- ▶ Позволяют ограничить набор наследников класса
  - ▶ справедливо и для интерфейсов
- ▶ По умолчанию – абстрактные и открыты для наследования
- ▶ Прямые наследники должны быть в одном пакете с **sealed** классом/интерфейсом
  - ▶ конструкторы могут быть только **protected** или **private**
- ▶ Преимущество при работе с **when** – выражением:
  - ▶ можно не ставить **else**, если покрыты все варианты

```
//по умолчанию абстрактный
sealed class Animation {
    abstract fun run()
}
//прямые наследники должны быть в том же пакете
class AttackAnimation : Animation() {
    override fun run() {
        println("hit some character")
    }
}
class WalkAnimation: Animation() {
    override fun run() {
        println("change legs and wave arms")
    }
}
//позволяет использовать when для быстрой проверки типа без else
fun peekAnim(animation: Animation) {
    val a: String = when(animation) {
        is AttackAnimation -> "Attack"
        is WalkAnimation -> "Walk"
    }
}
```

# Классы значений. Inline classes

## Проблема:

Таблица NPC

| id | type       |
|----|------------|
| 32 | peaceful   |
| 22 | aggressive |

```
fun getBanditById(id: Long) {  
    //при попытке передать id мирного жителя,  
    //нас никто не остановит  
}
```

## Решение:

```
//kotlin style  
inline class BanditId(val id: Long)  
  
fun getBanditById(id: BanditId) {  
    //вот теперь дамблатор не пропустит никого,  
    //так как BanditId превращается в Long  
} //но мы теряем оптимизацию Long, т.к.  
    //создается обёртка  
}
```

# Классы значений

## Inline classes (Kotlin 1.4)

- ▶ не больше одного параметра;
- ▶ никаких теневых полей;
- ▶ никаких блоков инициализации;
- ▶ никакого наследования.

## Value classes (Kotlin 1.5)

- ▶ Более широкий концепт, для будущих оптимизаций
- ▶ Поддержка init-блоков
- ▶ Изменение синтаксиса

```
@JvmInline  
value class BanditId(val id: Long)
```

# Any/ Unit / Nothing

- ▶ **Any** — стандартный родительский класс для всех классов Kotlin.
  - ▶ в отличии от **java.lang.Object** содержит только 3 метода
    - ▶ **equals(Object obj)** – возвращает true в случае, когда равны значения объекта, из которого вызывается метод, и объекта, передаваемого через ссылку obj в списке параметров.
    - ▶ **hashCode()** – выдаёт *хэш-код* объекта. В Java хэш-код заменяет адрес объекта в тех случаях, когда для каких-либо целей надо хранить таблицы адресов объектов.
    - ▶ **toString()** - Возвращает строковое представление объекта
- ▶ **Unit** — объект синглетон, применяется в случае, когда функция не имеет возвращаемого значения
- ▶ **Nothing** — наследник любого класса. Описывает результат функции, которая никогда ничего не вернёт.

# Nothing

- ▶ Тип, который не может появиться в программе
- ▶ Код после первого появления **Nothing** – недостижим
- ▶ Если ваша функция возвращает **Nothing**, это необходимо указать явно
  - ▶ Такая функция, с точки зрения компилятора, не завершается
  - ▶ Это очень важная информация, и компилятор просит у вас явного подтверждения, что это не ошибка, а намерение

```
//пример применения Nothing
public fun TODO(): Nothing =
    throw NotImplementedError()

fun main(args: Array<String>) {
    val command = if (args.size > 0) args[0]
    else throw IllegalArgumentException()
    val name = parseCommand(args) ?: return
    // ...
}
```

# План урока

- ▶ Коротко об ООП
- ▶ Структура класса
- ▶ Наследование и прочие родственные связи
- ▶ **Шаблоны проектирования и Kotlin**

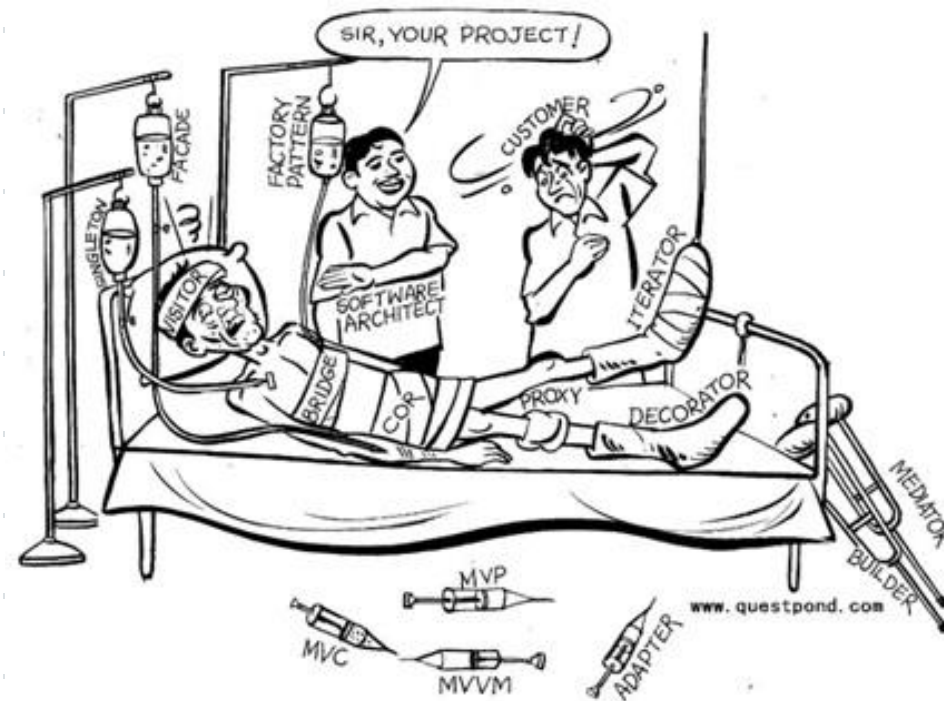


# Шаблоны проектирования

называют решения часто встречающихся проблем в области разработки программного обеспечения

## Распространенные шаблоны:

- ▶ Старый добрый Java-объект (POJO)
- ▶ Строитель (Builder)
- ▶ Делегирование (Delegation)
- ▶ Фабричный метод (Factory Method)
- ▶ Одиночка (Singleton)
- ▶ Цепочка обязанностей (Chain of responsibility)
- ▶ Стратегия (Strategy)
- ▶ ...



# POJO vs data class

## Plain Old Data Object

```
public class Point {  
    private final int x;  
    private final int y;  
  
    /*Подождите! Нам нужны:  
    * конструктор  
    * getX() и getY()  
    * equals и hashCode  
    * toString  
    * 40 строчек кода из ничего!  
    */
```

```
    public double distance(Point other) {  
        ...  
    }  
}
```

```
data class Point(val x: Int, val y: Int) {  
    //внутри уже готовые:  
    //getter & setter (ну они и до этого были)  
    //equals()/hashCode()  
    //toString()  
    //componentN()  
    //copy()  
    fun distance(other: Point) : Double {  
        TODO()  
    }  
}
```

# Data classes

## Требования:

- ▶ Основной конструктор должен иметь как минимум один параметр
- ▶ Все параметры основного конструктора должны быть отмечены, как **val** или **var**
- ▶ Классы данных не могут быть абстрактными, **open**, **sealed** или **inner**

Чтобы исключить свойство из автоматически созданной реализации, объявите его в теле класса:

```
data class NPC(val name: String) {  
    var hp: Int = 50  
}
```

# Мульти-декларация

## Деструктуризация

При создании **data-классов** компилятор автоматически добавляет набор функций, с помощью которых можно обратиться к свойствам.

Они известны как **componentN**-функции, где **N** - это номер свойства.

```
data class Item(val title: String, val type: TypeEnum, val cost: Int) {
    fun main() {
        val items = listOf(
            Item("Helm of Destiny", TypeEnum.HEAD, 20),
            Item("Shield of Protect", TypeEnum.ARMS, 30),
            Item("Shield of Protect", TypeEnum.ARMS, 30),
            Item("New shield component1", TypeEnum.ARMS, 45))
        //val type = shield.component2()
        //val cost = shield.component3()
        for ((title, _, cost) in items) {
            println("$title стоит $cost золотых")
        }
    }
}
```

- ▶ Этот механизм позволяет использовать деструктуризацию объекта
- ▶ Если “разобрать” не получается, компилятор выдаст ошибку
- ▶ Удобно использовать в циклах
- ▶ Любую переменную можно пропустить используя “\_”
- ▶ Работает только с классами, у которых определён **component**
- ▶ Хорошо работает с **data классами** и **map**

# Pair & Triple

## Pair

- ▶ data- класс, содержащий пару любых объектов
- ▶ Конструктор: `Pair(One, Two)`
  - ▶ или инфиксная функция `to`
    - ▶ `val cards = "Ace" to 11`
    - ▶ удобно использовать в деструктуризации
      - ▶ `val (name, value) = cards`

## Triple

- ▶ Используется 3 значения
- ▶ Удобен для дат
- ▶ Создаётся только конструктором



# Объекты. Анонимные объекты

## Object expressions

- ▶ Экземпляры анонимных классов, применяемых по месту
- ▶ Декларируются при помощи **“object”**
- ▶ Можно объявлять с нуля, наследовать существующие классы/реализовывать интерфейсы
- ▶ Если анонимный объект не имеет супертип, то его тип **Any**

```
interface A {  
    fun funFromA() {}  
}  
  
interface B  
  
class C {  
    // Возвращаемый тип Any. x недоступен  
    fun getObject() = object {  
        val x: String = "x"  
    }  
    // Возвращаемый тип A; x недоступен  
    fun getObjectA() = object: A {  
        override fun funFromA() {}  
        val x: String = "x"  
    }  
    // Возвращаемый тип B; funFromA() и x недоступны  
    fun getObjectB(): B = object: A, B {  
        override fun funFromA() {}  
        val x: String = "x"  
    }  
}
```

# Объекты. Singleton

## Object declarations

**Singleton** — порождающий шаблон проектирования, гарантирующий, что в однопоточном приложении будет единственный экземпляр некоторого класса, и предоставляющий глобальную точку доступа к этому экземпляру.

```
// Kotlin style
object Singleton {
    private Singleton() {}
    val a: Int = 42
    fun sayA() {}
}

// Java style
public class SingletonHolder {
    private static final Singleton INSTANCE = new Singleton();
}

public static Singleton getInstance() {
    return SingletonHolder.INSTANCE;
}

fun main() {
    SingletonHolder.INSTANCE.sayA()
}
```



# Делегирование

## Abstraction by Decorator Pattern

основной шаблон проектирования, в котором объект внешне выражает некоторое поведение, но в реальности передаёт ответственность за выполнение этого поведения связанному объекту.

```
//связываем реализацию коллекции с методами от innerList  
//аргументом передается тот же объект, который мы создаем это как обычно  
class DelegatingCollection<T> : Collection<T> {  
    (innerList: Collection<T>): Collection<T> by innerList {}  
    override val size: Int get() = innerList.size  
    override fun isEmpty(): Boolean = innerList.isEmpty()  
    ...  
}
```

- ▶ Делегировать можно не только классы, но и свойства
  - ▶ делегат должен предоставить реализацию **get/set**

# Practice 03 | Objects

1. Objects
2. Data classes



# Итог

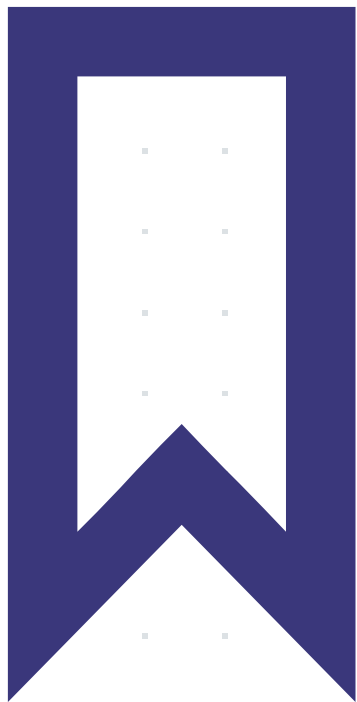
- ❑ ООП как в Java
  - ❑ Наследование, полиморфизм, инкапсуляция, вот это всё
- ❑ Есть поля на стероидах – свойства
- ❑ Конструкторы:
  - ❑ Основные\дополнительные
- ❑ Компаньоны – объекты, представляющие из себя статик члены класса
- ❑ Всё изначально закрыто и финализированно
- ❑ Any – родитель всех. Nothing – потомок всех
- ❑ Есть классы на уровне языка с поддержкой популярных фич



# Q&A



## Ещё по этой теме



### Ресурсы:

- ▶ <https://kotlinlang.ru/> - русскоязычная документация
- ▶ <https://play.kotlinlang.org/> - котлин онлайн
- ▶ <https://compscicenter.ru/courses/kotlinprogramming/>  
- курс Kotlin от CS center
- ▶ <https://kotlinlang.org/> - англоязычная документация

# Спасибо!



Aleksandr.Mityasov



@mityasov



Aleksandr.Mityasov@nexign.com

В настоящем документе содержится не предназначенная для публикации конфиденциальная информация и материалы, принадлежащие АО «Нэксайн» или его аффилированным лицам, охраняемые российским и международным законодательством об авторском, патентном праве, праве на секрет производства (ноу-хау), о товарных знаках и иных средствах индивидуализации.

Программное обеспечение АО «Нэксайн» и связанная с ним документация могут распространяться и использоваться только на основании отдельного лицензионного соглашения с АО «Нэксайн».

Любое использование, копирование, воспроизведение, публикация, обнародование, передача, изменение, переработка или перевод настоящего документа и содержащихся в нем информации и материалов, полностью или частично, в любой форме и любыми средствами без предварительного письменного разрешения АО «Нэксайн» строго запрещено.

Если настоящий документ предоставляется с программным обеспечением, лицензированным АО «Нэксайн», информация в нем предоставляется в соответствии с условиями гарантии, предоставляемой с лицензией на программное обеспечение.

Если настоящий документ не предоставляется с лицензионным программным обеспечением, информация в нем предоставляется «как есть» без каких-либо гарантий.

В обоих случаях АО «Нэксайн» не предоставляет никаких иных гарантий в отношении настоящего документа и содержащихся в нем информации и материалов, явных или подразумеваемых, включая, помимо прочего, подразумеваемые гарантии в отношении их качества, коммерческой пригодности, пригодности для определенной цели, ненарушения прав третьих лиц.

АО «Нэксайн» или его аффилированные лица ни при каких обстоятельствах не несут ответственность за любые прямые или косвенные убытки, случайные расходы или штрафы, возникшие в результате использования или невозможности использования материалов и/или информации, содержащихся в настоящем документе.

АО «Нэксайн» принимает надлежащие меры для обеспечения качества информации и материалов, содержащихся в настоящем документе. Однако документ может содержать технические неточности или опечатки, отсутствие ошибок не гарантируется.

Настоящий документ может быть изменен или дополнен без какого-либо уведомления. АО «Нэксайн» не обязуется обновлять настоящий документ на основе изменений в продуктах или услугах, как собственных, так и третьих лиц. АО «Нэксайн» может вносить улучшения или изменения в продукты или услуги, описанные в настоящем документе, в любое время без предварительного уведомления.



# Функциональщина и лямбда-ведение



# План урока

- ▶ Кое-что ещё о функциях
- ▶ Функциональные типы и лямбды
- ▶ Функции области видимости
- ▶ Последовательности

# План урока

- ▶ Кое-что ещё о функциях
- ▶ Функциональные типы и лямбды
- ▶ Функции области видимости
- ▶ Последовательности

# Что мы уже знаем про функции?

- ▶ Могут быть вне класса
- ▶ По умолчанию **final**
- ▶ Могут быть переопределены (**override**) и перегружены (**overload**)
- ▶ Чаще всего срабатывает **smartcast** и возвращаемый тип определяется сам
  - ▶ но иногда требуется писать самому (например: **Nothing**)
- ▶ **Unit** вообще писать не нужно
  - ▶ а если действие одно, то и **return** можно опустить и просто поставить “=”

```
fun checkInventory(player: Player) =  
    "у вас ${player.money} монет"
```



# Функции верхнего уровня

- ▶ На самом деле являются объектами со статическими методами
- ▶ Имя класса создаётся по имени файла

```
//kotlin file: Game.kt
fun startAdventure() {...}

//java-code
import package.Game
...
Game.startAdventure()
```

```
@file:JvmName("GameStarter")
fun `my#cu$t0m_nam3`() {
    TODO()
}
```

- ▶ Если имя класса вас не устраивает, то добавьте аннотацию `@JvmName` перед именем пакета.
  - ▶ а если в имени функции используются нестандартные символы, можно поместить её в обратные кавычки `

# Перегрузка операторов

Для определения оператора для типа определяется функция с ключевым словом **operator**:

```
operator fun название_оператора([параметры_оператора]) : возвращаемый_тип{  
    // действия функции оператора  
}
```

```
class Coins(val value: Int)  
  
fun main() {  
    val playerInventory = Coins(20)  
    val merchantInventory = Coins(100)  
    //так нельзя. + и - работают только для базовых типов  
    //или нет?  
    fun buyItem(cost: Coins) {  
        playerInventory = playerInventory - cost  
        merchantInventory = merchantInventory + cost  
    }  
}
```

```
operator fun plus(coins: Coins): Coins {  
    return Coins(this.value + coins.value)  
}  
operator fun minus(coins: Coins): Coins {  
    return Coins(this.value - coins.value)  
}
```

# Перегрузка операторов

| Выражение | Транслируется в |
|-----------|-----------------|
| +a        | a.unaryPlus()   |
| -a        | a.unaryMinus()  |
| !a        | a.not()         |
| a++       | a.inc()         |
| a--       | a.dec()         |
| a + b     | a.plus(b)       |
| a - b     | a.minus(b)      |
| a * b     | a.times(b)      |
| a / b     | a.div(b)        |
| a % b     | a.rem(b)        |
| a..b      | a.rangeTo(b)    |
| a in b    | b.contains(a)   |
| a !in b   | !b.contains(a)  |
| a[i]      | a.get(i)        |
| a[i] = b  | a.set(i, b)     |

| Выражение | Транслируется в                 |
|-----------|---------------------------------|
| a()       | a.invoke()                      |
| a(i)      | a.invoke(i)                     |
| a += b    | a.plusAssign(b)                 |
| a -= b    | a.minusAssign(b)                |
| a *= b    | a.timesAssign(b)                |
| a /= b    | a.divAssign(b)                  |
| a %= b    | a.modAssign(b)                  |
| a == b    | a?.equals(b) ?: (b === null)    |
| a != b    | !(a?.equals(b) ?: (b === null)) |
| a > b     | a.compareTo(b) > 0              |
| a < b     | a.compareTo(b) < 0              |
| a >= b    | a.compareTo(b) >= 0             |
| a <= b    | a.compareTo(b) <= 0             |

# Infix functions

Функции, помеченные ключевым словом **infix**, могут вызываться с использованием инфиксной записи (без точки и скобок для вызова)

```
class PlayerInventory(var coins: Coins) {  
    //инфикс-функцию можно определить только как член класса или  
    как расширение  
    infix fun buyItem(item: Item) {  
        coins = coins - item.cost  
    }  
    fun main() {  
        val playerInventory = PlayerInventory(Coins(20))  
        val helmet = Item("Iron Helmet", TypeEnum.HEAD, 10)  
        playerInventory buyItem helmet  
        //то же самое  
        playerInventory.buyItem(helmet)  
    }  
}
```

- ▶ Они должны являться членом класса или расширением
- ▶ В них должен использоваться только один параметр
- ▶ Параметр не должен принимать переменное количество аргументов и не должен иметь значения по умолчанию
- ▶ Вызовы инфиксных функций имеют более низкий приоритет, чем арифметические операторы

# Расширения

## Extension functions/properties

Расширения классов без наследования

```
fun тип.имя_функции(параметры) : возвращаемый_тип{  
    тело функции  
}
```

- ▶ this относится к объектам расширяемого типа
- ▶ Не являются членами класса
- ▶ Если у класса есть метод с той же сигнатурой, приоритет будет у него
- ▶ Аналогично функциям, есть свойства – расширения

```
fun String.deleteSpaces(): String {  
    return this.replace(" ", "")  
}   
val List<Int>.lastIndex: Int  
} get() = size - 1  
//используем функцию, как будто она член класса  
println("Hel lo, Wor ld".deleteSpaces()) // Hello,World
```



```
//java - code  
private static final String deleteSpaces(@NotNull String receiver) {  
    return receiver.replace(" ", "");  
}
```

# Функции с хвостовой рекурсией

```
private static final double eps = 1.0E-10D;
```

```
public static final double getEps() {  
    return eps;  
}
```

```
public static final double findFixPoint(double x) {  
    while(!(Math.abs(x - Math.cos(x)) < eps)) {  
        x = Math.cos(x);  
    }  
  
    return x;  
}
```

циклические  
иска переполнения

епа



```
val eps = 1E-10
```

```
tailrec fun findFixPoint(x: Double = 1.0): Double =  
    if (Math.abs(x - Math.cos(x)) < eps)  
        x  
    else  
        findFixPoint(Math.cos(x))
```

# Practice 01 | Functions

1. Overriding
2. Infix
3. Extention



# План урока

- ▶ Кое-что ещё о функциях
- ▶ **Функциональные типы и лямбды**
- ▶ Функции области видимости
- ▶ Последовательности

# Тип функций

Какой тип у этих функций?

```
fun doSomeAction(npc: NPC, animation: Animation) {  
    npc.walk()  
    animation.run() → Unit()  
}  
  
fun changeMoneyToCrystal(coins: Int) = coins/5 → Int()
```

Синтаксис :: - это ссылка на функцию, объявленную вне класса

Можем подставить функцию в качестве типа!

```
fun changeMoneyToCrystal(coins: Int) = coins/5  
fun changeCrystalToMoney(crystals: Int) = crystals*5  
  
fun main() {  
    //присваиваем переменной функциональный тип  
    var crystals: (Int) -> Int  
    //подставляем любую подходящую под описание функцию  
    crystals = ::changeMoneyToCrystal  
    //вызываем по имени переменной  
    crystals(10)  
    //можем динамически менять функцию  
    crystals = ::changeCrystalToMoney  
}
```

# Функции высшего порядка

функции, которые либо принимают функцию в качестве параметра, либо возвращают функцию, либо и то, и другое.

## Способы передачи функционального типа:

- ▶ лямбда-выражение
- ▶ анонимная функция
- ▶ ссылка на существующее объявление
  - ▶ `::` на функцию верхнего уровня
  - ▶ `this::` на метод класса
  - ▶ на свойства верхнего уровня, члена или свойства-расширения
  - ▶ на конструктор
- ▶ экземпляр класса, реализующий функциональный тип

```
fun changeMoneyToCrystal(coins: Int) = coins/5
fun changeCrystalToMoney(crystals: Int) = crystals*5

fun main() {
    //У игрока 120 монет
    val player = Player("Alex", PlayerClasses.WARRIOR, 50, 100,
        PlayerInventory(Coins(120)))
    //вызываем от функцию с параметром м->к
    player.inventory.transaction(::changeMoneyToCrystal)
    println(player.inventory.coins.value) //24 (120/5)
}
//делаем функцию-расширение для инвентаря
//она будет принимать на вход функцию обмена к->м или м->к
fun PlayerInventory.transaction(t: (Int)-> Int) {
    coins = when(t) {
        //вызов функционального типа - t.invoke() или t() т.к. это оператор
        ::changeMoneyToCrystal -> Coins(t.invoke(coins.value))
        ::changeCrystalToMoney -> Coins(t(coins.value))
        else -> {throw RuntimeException()}
    }
}
```

# Лямбда выражения

это функция, записанная в виде выражения, и которую можно передавать как аргумент в другие функции.

**{имяАргумента: Тип -> телоФункции}**

- ▶ Лямбда-выражение всегда заключено в скобки {...};
- ▶ Объявление параметров при таком синтаксисе происходит внутри этих скобок и может включать в себя аннотации типов;
- ▶ Тело функции начинается после знака ->;
- ▶ Если тип возвращаемого значения не Unit, то в качестве возвращаемого типа принимается последнее (а возможно и единственное) выражение внутри тела лямбды.

```
//функция высшего порядка,  
// т.к. принимает ф-ю в качестве аргумента  
max(strings, { a, b -> a.length < b.length })  
  
//определять тип не обязательно  
val sum: (Int, Int) -> Int = { x: Int, y: Int -> x + y }  
  
//как и не обязательно указывать его в лямбде  
player.inventory.transaction { x -> x / 5 }
```

# Лямбда выражения

```
var zero = {println("без аргументов")}
//zero - ничего не выведет
//а так, работает
zero()
//у лямбды один параметр
var one: (String) -> String = {it.uppercase()}
//то же что и
one = {s -> s.uppercase()}
//вынесли лямбду за скобки, хоть она и аргумент
val two = lambdaTest(4, "test") {it.length}
val map = mapOf(Pair(4, "четыре"), Pair(3, "три"))
//пропускаем ненужные параметры
val three = map.forEach { (_, value) -> println("$value!") }
map.mapValues { entry -> "${entry.value}!" }
// внимание! два параметра - { a, b -> ... }
// пара дедектурирования - { (a, b) -> ... }
map.mapValues { (key, value) -> "$key $value!" }
//захват значения count
var count = 0
val sum = { x: Int, y: Int -> count++
    x + y }
```

- ▶ Если выражение имеет один параметр, его можно заменить на **it**
- ▶ Если последний параметр функции является функцией, то лямбда-выражение, переданное в качестве соответствующего аргумента, может быть вынесено за круглые скобки
- ▶ Если параметр лямбды не используется, то разрешено его имя заменить на символ подчёркивания
- ▶ Можно использовать при дедектуризации
- ▶ Лямбды могут захватывать переменные вне выражения
- ▶ Лямбды возвращают значение последнего выражения.

**Return** не нужен!

- ▶ но если очень хочется, то можно:
  - ▶ **return@lambdaName result**

# Анонимные функции

Вариант синтаксиса, где имя функции не указывается.

Могут применяться так же, как и лямбды

```
fun high(f: (Int, Int) -> Int, x: Int, y: Int) {  
    println(f(x,y))  
}  
val lambdaReturn = high({x, y -> return@high x * y}, 5,6)  
  
val anonimReturn = high(  
    fun(x:Int, y:Int): Int {  
        return x * y }, 5,6)
```

## Применимость:

- ▶ Когда нужно явным образом указать возвращаемый тип
- ▶ Когда требуется использовать большой блок кода с **return**
  - ▶ да, в анонимных функциях, **return** работает как обычно

# Функциональные (SAM) интерфейсы

Интерфейсы только с одним абстрактным методом называются функциональными интерфейсами или Single Abstract Method (SAM) интерфейсами.

Функциональный интерфейс может иметь несколько неабстрактных членов, но только один абстрактный.

## Применимость:

- ▶ Когда для вычисления абстрактной функций требуется вспомогательные данные (функции по умолчанию)
- ▶ Когда в функции API определено строгое наследование

```
//интерфейс с одним абстрактным методом
fun interface IntPredicate {
    fun accept(i: Int): Boolean
}

// Создание экземпляра класса
val isEven = object : IntPredicate {
    override fun accept(i: Int): Boolean {
        return i % 2 == 0
    }
}

// Создание экземпляра, используя лямбду
val isEven = IntPredicate { it % 2 == 0 }
```

# Встроенные функции

## inline functions

Правило простое: если ваш код множество раз вызывает функцию, принимающую лямбду в качестве аргумента, лучше сделать ее **inline**-функцией.

- ▶ Компилятор будет пробовать встраивать тело такой функции прямо в место вызова
- ▶ Более того, он будет пробовать встраивать и переданные в **inline**-функцию лямбда-аргументы
- ▶ Это позволяет улучшить производительность за счет некоторых дополнительных оптимизаций
- ▶ Лямбда-аргументы у **inline**-функции встраиваются по умолчанию
- ▶ Если мы хотим это запретить, используем **noinline** на аргументе

```
public inline fun <T, R> T.run(block: T.() -> R): R {  
    return block()  
}
```

```
class FunctionStorage(  
    val functionResults: MutableMap<() -> Any, Any>)  
inline fun FunctionStorage.run(noinline body: () -> Any) =  
    functionResults[body] ?: kotlin.run {  
        val res = body()  
        functionResults[body] = res  
        res  
    }
```

# Функциональные типы с неявным this

Функциональные типы с получателем, такие как **A.(B) -> C**, могут быть вызваны с помощью особой формы – литералов функций с объектом-приёмником.

```
//T,R - параметризованные типы:  
//определяются по месту использования  
fun <T, R> with(receiver: T, block: T.() -> R): R =  
    receiver.block()
```

```
//main style  
fun buildPlayer() Player {  
    with(PlayerBuilder()) {  
        setName("Alex")  
        setCoin(50)  
        setHp(100)  
    }  
}  
public void setName(final String name) {}  
public void setHp(final int hp) {}  
public void setCoin(final int coin) {}  
} public Player build() {}
```

# План урока

- ▶ Кое-что ещё о функциях
- ▶ Функциональные типы и лямбды
- ▶ **Функции области видимости**
- ▶ Последовательности

# Функции области видимости

run, also, apply, with, let



- ▶ ... и они почти одинаковы!
- ▶ Призваны выполнять блок кода на объекте
- ▶ Просто добавляют сахар
- ▶ Различия:
  - ▶ Способ ссылки на контекстный объект
  - ▶ Возвращаемое значение
    - ▶ **apply** и **also** возвращают объект контекста.
    - ▶ **let**, **run** и **with** возвращают результат лямбды.

# Функции области видимости

## let

```
public inline fun <T, R> T.let(block: (T) -> R): R {  
    return block(this)  
}  
  
fun processNonNullString(str: String) {}  
  
fun main() {  
    val str: String? = "Hello"  
    //processNonNullString(str)    // compilation error: str может быть null  
    val length = str?.let {  
        println("Вызов функции let() для $it")  
        // OK: 'it' не может быть null внутри конструкции '?.let { }'  
        processNonNullString(it)  
        it.length  
    }  
}
```

- ▶ Контекстный объект доступен в качестве аргумента (it)
- ▶ Возвращаемое значение - результат выполнения лямбды.
- ▶ **let** часто используется для выполнения блока кода только с non-null значениями

# Функции области видимости

## with

```
fun main() {  
    val numbers = mutableListOf("one", "two", "three")  
    with(numbers) {  
        println("'with' вызывает с аргументом $this")  
        println("Список содержит $size элементов")  
    }  
}
```

- ▶ Не является функцией-расширением
- ▶ Контекстный объект передается в качестве аргумента
  - ▶ внутри лямбда-выражения он доступен как получатель (**this**)
- ▶ Возвращаемое значение - результат выполнения лямбды
- ▶ В коде **with** может читаться как *"с этим объектом, сделайте следующее."*

# Функции области видимости

## run

```
class Main() {  
    portService(var url: String, var port: Int) {  
        fun prepareRequest() String = "Запрос по умолчанию"  
        fun query(request: String): String = "Результат: '$request'"  
    }  
    val hexDigits = "A-Fa-f"  
    val sign = "+-"  
    fun main() {  
        val RegEx = "$Sign*$hexDigits*?example.org", 80)  
    }  
    val result = service.run {  
        port = 8080  
        query(prepareRequest() + " порт - $port")  
    }  
}
```

- ▶ Контекстный объект доступен в качестве получателя (**this**)
- ▶ Возвращаемое значение - результат выполнения лямбды
- ▶ **run** делает то же самое, что и **with**
  - ▶ но вызывается как **let**
- ▶ **run** удобен, когда лямбда содержит и инициализацию объекта, и вычисление возвращаемого значения
- ▶ можно использовать без расширения
  - ▶ в таком случае, просто выполняет блок операторов

# Функции области видимости

## apply

```
data class Person(var name: String,
                  var age: Int = 0,
                  var city: String = "")
fun main() {
    val adam = Person("Adam").apply {
        age = 32
        city = "London"
    }
    println(adam)
}
```

- ▶ Контекстный объект доступен в качестве получателя (**this**)
- ▶ Возвращаемое значение - контекстный объект
- ▶ Типичный способ применения – настройка объекта получателя
- ▶ Т.к. возвращаемое значение – объект, то **apply** удобно использовать в цепочках

# Функции области видимости

## also

```
fun main() {  
    val numbers = mutableListOf("one", "two", "three")  
    numbers  
        .also { println("Элементы списка перед добавлением: $it") }  
        .add("four")  
}
```

- ▶ Контекстный объект доступен в качестве аргумента (**it**)
- ▶ Возвращаемое значение - контекстный объект
- ▶ подходит, когда требуется ссылка на сам объект
- ▶ можно прочитать как *"а также с объектом нужно сделать следующее."*

# Выбор функции

| Функция | Обращение к объекту | Возвращаемое значение | Является функцией-расширением                           |
|---------|---------------------|-----------------------|---------------------------------------------------------|
| let     | it                  | Результат лямбды      | Да                                                      |
| run     | this                | Результат лямбды      | Да                                                      |
| run     | -                   | Результат лямбды      | Нет: может быть вызвана без контекстного объекта        |
| with    | this                | Результат лямбды      | Нет: принимает контекстный объект в качестве аргумента. |
| apply   | this                | Контекстный объект    | Да                                                      |
| also    | it                  | Контекстный объект    | Да                                                      |

## Что выбрать?

- ▶ Выполнение лямбды для **non-null** объектов: **let**
- ▶ Представление переменной в виде выражения со своей локальной областью видимости: **let**
- ▶ Настройка объекта: **apply**
- ▶ Настройка объекта и вычисление результата: **run**
- ▶ Выполнение операций, для которых требуется выражение: **run** без расширения
- ▶ Применение дополнительных значений: **also**
- ▶ Группировка всех функций, вызываемых для объекта: **with**

# Practice 02 | Lambdas

1. Lambda
2. scope functions



# План урока

- ▶ Кое-что ещё о функциях
- ▶ Функциональные типы и лямбды
- ▶ Функции области видимости
- ▶ **Последовательности**



# Sequences

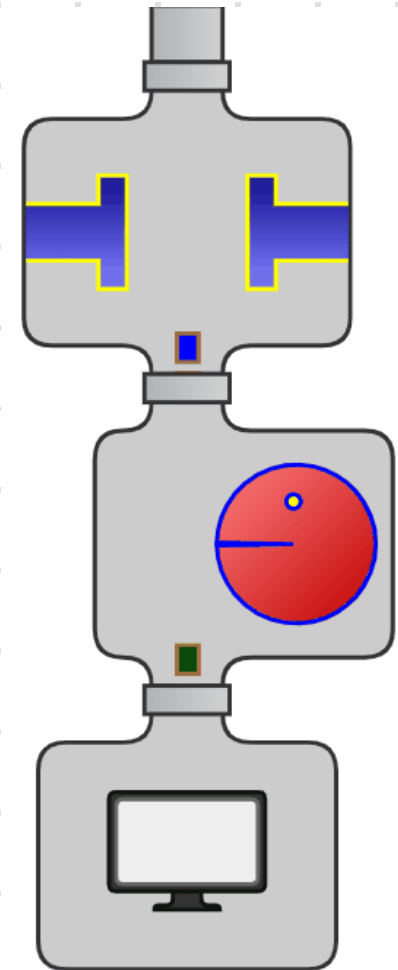
В Java 8 появились стримы — лениво вычисляемые потоки данных

**Stream** - последовательность элементов (возможно, бесконечная) с поддержкой различных преобразований

- ▶ Декларативный подход к обработке данных: описываем что хотим получить, а не как мы это получим
- ▶ Может потребить не все элементы

В Kotlin аналогичная штука: **Sequence**

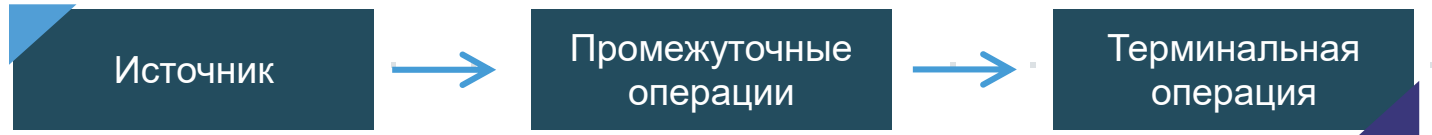
- ▶ Ленивый аналог Iterable
- ▶ Работают как в java



# Sequences

## Создание:

- ▶ С помощью элементов
- ▶ С помощью Iterable
- ▶ С помощью функции
- ▶ С помощью чанков (chunks)



```
val numbersSequence = sequenceOf("four", "three", "two", "one")
val numbers = listOf("one", "two", "three", "four")
val numbersSequence = numbers.asSequence()
val oddNumbers = generateSequence(1) { it + 2 } // `it` - это предыдущее значение
println(oddNumbers.take(5).toList()) // [1, 3, 5, 7, 9]
//println(oddNumbers.count()) // ошибка: последовательность бесконечна
val oddNumbers = sequence {
    yield(1)
    yieldAll(listOf(3, 5))
    yieldAll(generateSequence(7) { it + 2 })
}
println(oddNumbers.take(5).toList()) // [1, 3, 5, 7, 9]
```

# Practice 03 | Sequences



# Итог

- ❑ Функции – полноценные сущности и это здорово!
  - ❑ можно расширять любые классы (даже недоступные)
  - ❑ можно записывать в инфиксном виде
  - ❑ можно даже перегрузить операторы!
- ❑ А ещё, их можно использовать в виде типов
  - ❑ тем самым, создавая функции высшего порядка!
- ❑ Создавать лямбды тоже удобно
  - ❑ хотя можно и по старинке, через интерфейсы
  - ❑ есть даже готовые лямбды на уровне языка, добавляющие сахара
- ❑ Последовательности – это стримы из java



# Q&A



# Спасибо!



Aleksandr.Mityasov



@mityasov



Aleksandr.Mityasov@nexign.com

В настоящем документе содержится не предназначенная для публикации конфиденциальная информация и материалы, принадлежащие АО «Нэксайн» или его аффилированным лицам, охраняемые российским и международным законодательством об авторском, патентном праве, праве на секрет производства (ноу-хау), о товарных знаках и иных средствах индивидуализации.

Программное обеспечение АО «Нэксайн» и связанная с ним документация могут распространяться и использоваться только на основании отдельного лицензионного соглашения с АО «Нэксайн».

Любое использование, копирование, воспроизведение, публикация, обнародование, передача, изменение, переработка или перевод настоящего документа и содержащихся в нем информации и материалов, полностью или частично, в любой форме и любыми средствами без предварительного письменного разрешения АО «Нэксайн» строго запрещено.

Если настоящий документ предоставляется с программным обеспечением, лицензированным АО «Нэксайн», информация в нем предоставляется в соответствии с условиями гарантии, предоставляемой с лицензией на программное обеспечение.

Если настоящий документ не предоставляется с лицензионным программным обеспечением, информация в нем предоставляется «как есть» без каких-либо гарантий.

В обоих случаях АО «Нэксайн» не предоставляет никаких иных гарантий в отношении настоящего документа и содержащихся в нем информации и материалов, явных или подразумеваемых, включая, помимо прочего, подразумеваемые гарантии в отношении их качества, коммерческой пригодности, пригодности для определенной цели, ненарушения прав третьих лиц.

АО «Нэксайн» или его аффилированные лица ни при каких обстоятельствах не несут ответственность за любые прямые или косвенные убытки, случайные расходы или штрафы, возникшие в результате использования или невозможности использования материалов и/или информации, содержащихся в настоящем документе.

АО «Нэксайн» принимает надлежащие меры для обеспечения качества информации и материалов, содержащихся в настоящем документе. Однако документ может содержать технические неточности или опечатки, отсутствие ошибок не гарантируется.

Настоящий документ может быть изменен или дополнен без какого-либо уведомления. АО «Нэксайн» не обязуется обновлять настоящий документ на основе изменений в продуктах или услугах, как собственных, так и третьих лиц. АО «Нэксайн» может вносить улучшения или изменения в продукты или услуги, описанные в настоящем документе, в любое время без предварительного уведомления.



# Дженерики и коллекции

# План урока

- ▶ Правда о коллекциях
- ▶ Типы коллекций
- ▶ «Обобщай и властвуй» или коротко о дженериках

# План урока

- ▶ **Правда о коллекциях** 
- ▶ Типы коллекций
- ▶ «Обобщай и властвуй» или коротко о дженериках

# Коллекции

это набор объектов, упорядоченный по определенным правилам

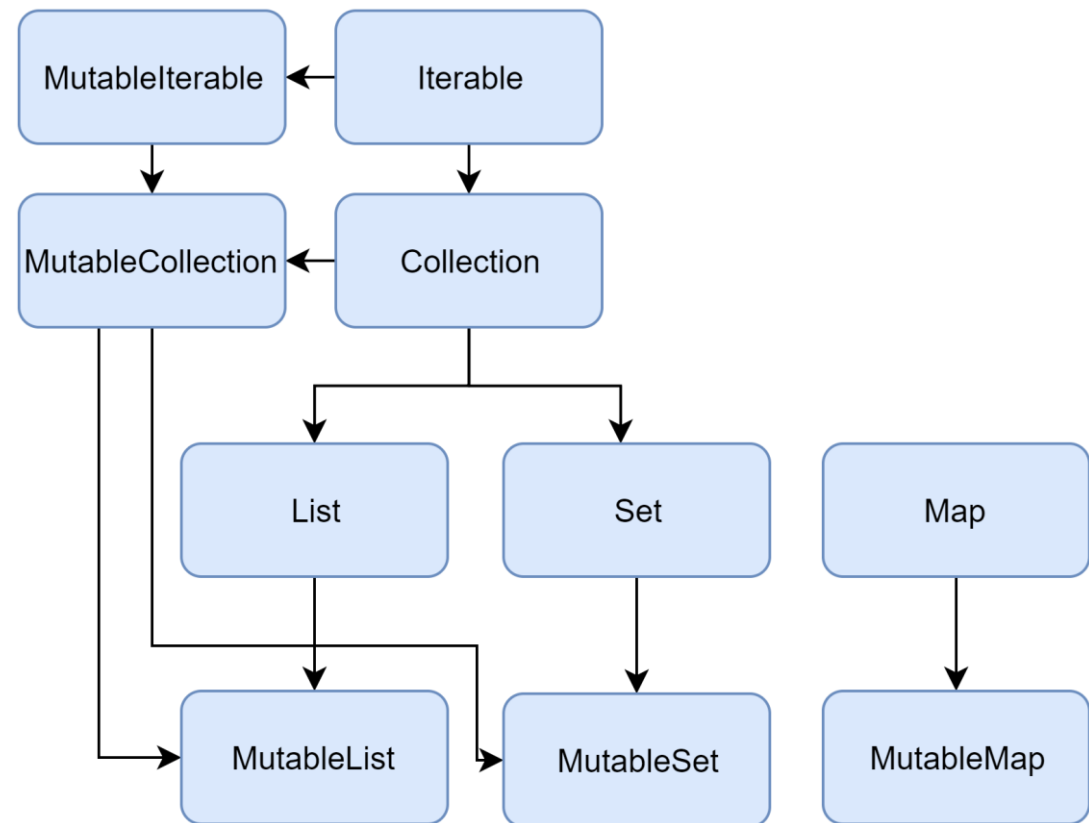
- ▶ Обёртки над Java коллекциями с доп. функциями (расширениями)
- ▶ Бывают изменяемые (**mutable**) и неизменяемые (**immutable**)
- ▶ Есть удобные функции для вызова
  - ▶ `listOf<>/mutableListOf<>()`
  - ▶ `setOf<>/mutableSetOf<>()`
  - ▶ `mapOf<>/mutableMapOf<>()`
- ▶ Есть возможность создать пустые коллекции (`emptyList()`, `emptySet()` и т.д.)



# Иерархия интерфейсов коллекций

- ▶ Неизменяемые типы коллекций **ковариантны**
- ▶ Изменяемые коллекции – **инвариантны**

```
//да  
val someInt: Int = 1  
val abstractNumber: Number = someInt  
//тоже да  
val immutableIntList: List<Int> = listOf<Int>(1, 2)  
val immutableNumbersList: List<Number> = immutableIntList  
//но  
val mutableStringList: MutableList<String>  
    = mutableListOf<String>("один", "два")  
val mutableAnyList: MutableList<Any> = mutableStringList
```



# Итератор

- ▶ Доступен всем коллекциям по функции **iterator()**
- ▶ Позволяет осуществлять проход по элементам коллекции
- ▶ Есть **ListIterator** для прохода вперёд и назад
  - ▶ **previous()**
  - ▶ **next()**
- ▶ Есть **MutableIterator** реализующий **remove()**

```
val numbers = listOf("one", "two", "three", "four")
//итератор
val numbersIterator = numbers.iterator()
while (numbersIterator.hasNext()) {
    println(numbersIterator.next())
}
//неявный итератор
for (item in numbers) {
    println(item)
}
//с помощью функции
numbers.forEach {
    println(it)
}
```

# Создание коллекций

- ▶ При помощи функций
- ▶ При помощи инициализатора для списков
- ▶ Конструкторами конкретных типов
- ▶ Копированием

```
//to() - инфиксная функция, которая создаёт объект Pair
val numbersMap = mapOf("key1" to 1, "key2" to 2, "key3" to 3, "key4" to 1)
val numbersStringMap = mutableMapOf<String, String>()
    .apply {
        this["one"] = "1"
        this["two"] = "2"
    }
val empty = emptyList<String>()
val doubled = List(3, { x -> x * 2 })
//кстати, тип коллекции указывать не обязательно
//Kotlin может вывести его автоматически из элементов
val linkedList = LinkedList<String>(listOf("one", "two", "three"))
val presizedSet = HashSet<Int>(32)
val copyList = linkedList.toMutableSet()
```

# План урока

- ▶ Правда о коллекциях
- ▶ **Типы коллекций**
- ▶ «Обобщай и властвуй» или коротко о дженериках

# Разновидности коллекций



## List

список (фиксированный  
порядок, доступ к  
элементам по индексу)



## Set

множество (каждый  
элемент встречается не  
более одного раза)



## Map

ассоциативный массив  
(набор пар  
«ключ—значение»)

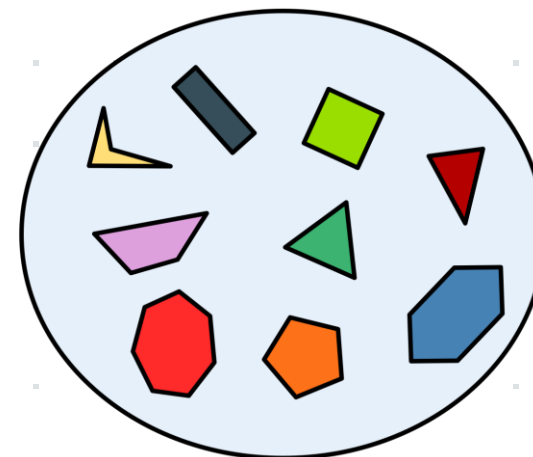
# List

- ▶ **List<> / listOf<>**
- ▶ Фиксированный порядок
- ▶ Доступ к элементам по индексу
  - ▶ **get()** или его перегруженный вариант [ ]
  - ▶ а ещё можно **getOrNull()** {} или **getOrElse()** {}
- ▶ списки равны, если содержат структурно равные элементы в одинаковом порядке
- ▶ **MutableList** добавляет методы-модификации
  - ▶ **add()**
  - ▶ **set()** или **mList[index] = newValue**
  - ▶ **remove()**
- ▶ реализация по умолчанию – **ArrayList**
  - ▶ массив с изменяемым размером



# Set

- ▶ **Set<>/ setOf<>()**
- ▶ Порядок не определён
- ▶ Каждый элемент встречается не более одного раза
- ▶ Множества равны, если содержат одинаковые элементы
- ▶ Может содержать ровно один **null**
- ▶ **MutableSet** добавляет операции записи
- ▶ Реализации:
  - ▶ **LinkedHashSet** – сохраняет порядок вставки
    - ▶ **first()/last()**
  - ▶ **HashSet** – неопределённый порядок



# Map

- ▶ **Map<>/ mapOf<>()**
- ▶ Набор пар «ключ-значение»
  - ▶ Ключи уникальны
- ▶ Не входит в иерархию коллекций
- ▶ Равны, если пары равны, независимо от порядка
- ▶ **MutableMap** добавляет операции изменения
  - ▶ **put()**
- ▶ Реализации по умолчанию:
  - ▶ **LinkedHashMap** – сохраняет порядок
  - ▶ **HashMap** – не упорядочен



# Операции с коллекциями

```
val names = listOf("Леголас", "Гендальф", "Арагорн")
val classes = listOf("лучник", "волшебник", "воин")
//zip - объединяет два списка в список Pair
val heroesMap = names.zip(classes).toMap()
println(heroesMap["Арагорн"])

val items = (1..5).toMutableList()
println("Было: $items")
//заполняет список нужным значением
items.fill(5)
println("Стало: $items")
//map применяет лямбду к каждому
//элементу и возвращает коллекцию
val square = items.map { it * it }
```

```
val npcs = listOf(
    NPC("Horn the Blacksmith", 150),
    NPC("Falgor", 120)
)
//associate{} - собрать Map из членов класса
val associatedMap = npcs.associate { Pair(it.name, it.hp) }
associatedMap.forEach { entry ->
    println("Use a ${entry.value} on a ${entry.key}")
}
//если ключ не найден, вернёт значение по умолчанию
associatedMap.getDefault("Aragorn", 200)
//цепочечный вызов
//фильтрует элементы и меняет имя
associatedMap
    .filter { entry -> entry.value > 150 }
    .map { entry -> "sir " + entry.key }
```

# План урока

- ▶ Правда о коллекциях
- ▶ Типы коллекций
- ▶ **«Обобщай и властвуй» или коротко о дженериках**



# Проблема

```
val bow: Weapon = ...  
val sword: Weapon = ...  
val book: Item = ...  
//поместили book хоть она и не weapon  
val weaponList: MutableList = mutableListOf(bow, sword, book)  
// ошибка типов, но слишком поздно  
val someWeapon: Weapon = weaponList.get(2)
```

# Решение

```
val bow: Weapon = ...  
val sword: Weapon = ...  
val book: Item = ...  
//ошибка book поместить уже нельзя  
val weaponList: MutableList<Weapon> = mutableListOf(bow, sword, book)
```



# Обобщения

## Generics

- ▶ Позволяют указать «условный тип», с которым работает код
- ▶ Используются, когда один и тот же код применим к большому числу типов
- ▶ Позволяют повысить безопасность, за счет контроля типов на стадии компиляции программы, а не выполнения
- ▶ Все интерфейсы и классы коллекций – параметризованные
- ▶ Параметр типа может использоваться внутри нестатических членов класса:
  - ▶ Тип поля, параметра, переменной
  - ▶ Возвращаемый тип метода
  - ▶ Подставляться параметрам другого типа

# Обобщения

## Generics

```
//параметризовать(обобщить) можно почти что угодно
class ValueWrapper<T>(val value: T) {
    fun <R> convertTo(): R = value as R
}
//тип параметр можно не указывать, Kotlin выведет его из контекста
val value = ValueWrapper(4)
```

- ▶ Нельзя создать объект обобщённого типа
- ▶ Нельзя проверять на соответствие типу (**is**)
- ▶ Использовать в **companion object**
  - ▶ а всё потому, что во время выполнения значения дженериков стираются (**type erasure**)
  - ▶ иными словами, везде где есть **T** – написано **Any?**

# Ограниченные типы

- ▶ Обобщённым типам можно задавать ограничение по наследованию
  - ▶ `class<T: ParentClass>`
  - ▶ класс может принять все подтипы **ParentClass**
    - ▶ при стирании типов, тип будет приведён к верхней границе
    - ▶ это позволяет пользоваться всеми методами и свойствами этого типа
- ▶ По умолчанию, можно указывать только одну границу
  - ▶ для указания нескольких границ (класс + интерфейс) используется условие **where**

```
fun <T : Comparable<T>> sort(list: List<T>) { ... }  
// Всё в порядке. Int — подтип Comparable<Int>  
sort(listOf(1, 2, 3))  
//T должен наследоваться от CharSequence и реализовывать Comparable  
fun <T> copyWhenGreater(list: List<T>, threshold: T): List<String> where T : CharSequence, T : Comparable<T> {  
    ...  
}
```

# Вариативность

## in/out variance

- ▶ Обобщения по умолчанию **инвариативны**
  - ▶ Это значит, что **List<String>** не является подтипом **List<Any>**
- ▶ Мы можем указать уровень вариантности вручную, добавляя указатели **in/out**:
  - ▶ **List<out T>** - делает классы ковариантными (классическая схема наследования типов)
  - ▶ Объекты данного типа считаются потребителями (**consumers**) и могут реализовывать только запись значений
  - ▶ **List<in T>** - делает классы контравариантными, т.е.
    - ▶ **List<String>** принимает **List<Any>**, но не наоборот
  - ▶ Объекты такого типа считаются производителями (**producers**) и могут только отдавать значения

```
//ошибка, типы инвариантны
interface MyList<T>
fun some(list: MyList<String>) {
    val anyList:MyList<Any> = list
}

//типы ковариантны, всё ок
interface MyList<out T>
fun some(list: MyList<String>) {
    val anyList:MyList<Any> = list
}

//типы контр вариантны, поменяли сигнатуру
interface MyList<in T>
fun some(list: MyList<Any>) {
    val anyList:MyList<String> = list
}
```

# Вариативность

## in/out variance

- ▶ На ковариантном параметре функции можно вызывать только
- ▶ методы, которые возвращают T
- ▶ На контравариантном параметре функции — по аналогии — можно
- ▶ вызывать только принимающие T методы
- ▶ **in/out** дженерики — аналог wildcards из Java
  - ▶ **out T** === ? extends T
  - ▶ **in T** === ? super T

# PICO

Producers - *in*, consumer - *out*

```
class MyList<out T> (private val list:MutableList<T>){  
    //тип ковариантен, добавлять нельзя  
    fun add(e: T) {  
        list.add(e)  
    }  
  
    class MyList<in T> (private val list:MutableList<T>){  
        //тип контрвариантен, читать нельзя  
        fun get(index: Int): T {  
            return list.get(index)  
        }  
    }  
}
```

# Star projection

- ▶ Иногда вы не знаете, что за тип у дженерика
- ▶ В Kotlin используются star-проекции **T<\*>**
  - ▶ **T<out E : UpperBound>**
    - ▶ читаем наиболее общий возможный тип
  - ▶ **T<in E : UpperBound>**
    - ▶ т.е. запись запрещена
  - ▶ **T<E : UpperBound>**
    - ▶ оба вместе

```
class MyList<out T> (private val list:MutableList<out T>){  
    fun get(index: Int): T {  
        return list.get(index)  
    }  
}  
//читаем в Any? как наиболее общий  
fun starFun(starList: MyList<*>): Any? {  
    return starList.get(2)  
}
```

# Reified generics

- ▶ В Kotlin есть **inline** функции
- ▶ Их тело встраивается в место вызова
  - ▶ мы можем сохранить фактический тип дженерика!
  - ▶ **<reify T>** - вытащить тип дженерика и подставить в место вызова
- ▶ **Работает только в inline!**

```
inline fun <reified R> List<Any>.removeAllInstancesOf()
    : List<Any> {
    var res = listOf<Any>()
    for (e in this) {
        //такие штуки с дженериками (is R) обычно запрещены
        //но для reified можно
        if (e !is R) res += e
    }
    return res
}

fun main() {
    val l = listOf(1, "", 5.0).removeAllInstancesOf<String>()
    println(l)
}
```

# Итог

- ❑ Коллекции Kotlin – не что иное как коллекции Java
  - ❑ создавать можно при помощи функций
  - ❑ бывают изменяемые и не очень
  - ❑ работает вывод типов
  - ❑ есть удобные методы для обработки (map/filter и т.д.)
- ❑ Обобщения – обобщают
  - ❑ работает стирание типов
  - ❑ по умолчанию инвариантны
  - ❑ работает PICO
  - ❑ а если не знаем тип обобщения, то ставим звезду



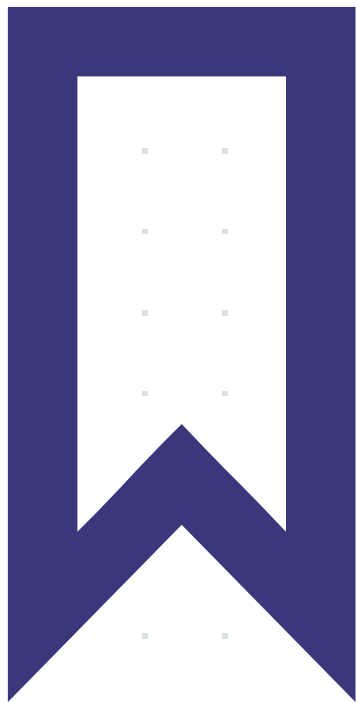
# Q&A



# Вопросы для самоконтроля

- ▶ В чём разница между изменяемыми и неизменяемыми коллекциями?
- ▶ Почему коллекции в Java и Kotlin инвариантны по типу значений?
- ▶ Для чего нужны обобщения?
- ▶ Как работают принципы вариативности типов?
- ▶ Для чего нужны «звёздные» обобщения?

## Ещё по этой теме



### Ресурсы:

- ▶ <https://kotlinlang.ru/> - русскоязычная документация
- ▶ <https://play.kotlinlang.org/> - котлин онлайн
- ▶ <https://compscicenter.ru/courses/kotlinprogramming/>  
- курс Kotlin от CS center
- ▶ <https://kotlinlang.org/> - англоязычная документация
- ▶ <https://habr.com/ru/company/maxilect/blog/590061/> - про “грабли” Kotlin

# Спасибо!



Aleksandr.Mityasov



@mityasov



Aleksandr.Mityasov@nexign.com

В настоящем документе содержится не предназначенная для публикации конфиденциальная информация и материалы, принадлежащие АО «Нэксайн» или его аффилированным лицам, охраняемые российским и международным законодательством об авторском, патентном праве, праве на секрет производства (ноу-хау), о товарных знаках и иных средствах индивидуализации.

Программное обеспечение АО «Нэксайн» и связанная с ним документация могут распространяться и использоваться только на основании отдельного лицензионного соглашения с АО «Нэксайн».

Любое использование, копирование, воспроизведение, публикация, обнародование, передача, изменение, переработка или перевод настоящего документа и содержащихся в нем информации и материалов, полностью или частично, в любой форме и любыми средствами без предварительного письменного разрешения АО «Нэксайн» строго запрещено.

Если настоящий документ предоставляется с программным обеспечением, лицензированным АО «Нэксайн», информация в нем предоставляется в соответствии с условиями гарантии, предоставляемой с лицензией на программное обеспечение.

Если настоящий документ не предоставляется с лицензионным программным обеспечением, информация в нем предоставляется «как есть» без каких-либо гарантий.

В обоих случаях АО «Нэксайн» не предоставляет никаких иных гарантий в отношении настоящего документа и содержащихся в нем информации и материалов, явных или подразумеваемых, включая, помимо прочего, подразумеваемые гарантии в отношении их качества, коммерческой пригодности, пригодности для определенной цели, ненарушения прав третьих лиц.

АО «Нэксайн» или его аффилированные лица ни при каких обстоятельствах не несут ответственность за любые прямые или косвенные убытки, случайные расходы или штрафы, возникшие в результате использования или невозможности использования материалов и/или информации, содержащихся в настоящем документе.

АО «Нэксайн» принимает надлежащие меры для обеспечения качества информации и материалов, содержащихся в настоящем документе. Однако документ может содержать технические неточности или опечатки, отсутствие ошибок не гарантируется.

Настоящий документ может быть изменен или дополнен без какого-либо уведомления. АО «Нэксайн» не обязуется обновлять настоящий документ на основе изменений в продуктах или услугах, как собственных, так и третьих лиц. АО «Нэксайн» может вносить улучшения или изменения в продукты или услуги, описанные в настоящем документе, в любое время без предварительного уведомления.



# Корутины и DSL



# План урока

- ▶ Проблемы многопоточности
- ▶ Реактивное введение в корутины
- ▶ DSL – это не модем

# План урока

- ▶ Проблемы многопоточности
- ▶ Реактивное введение в корутины
- ▶ DSL – это не модем

# Многопоточность

- ▶ Одновременное выполнение нескольких действий (например, отрисовка пользовательского интерфейса и передача файлов по сети)
- ▶ Ускорение вычислений (при наличии нескольких вычислительных ядер)

```
val image = loadImage(URL)  
myUI.setImage(image)
```



# Решения

- ▶ Callback
- ▶ Features/Promises
- ▶ Threads

**Thread** — составная часть процесса, который выполняется в операционной системе.

- ▶ Потоками управляет операционная система
- ▶ Переключение потоков сложно контролировать
- ▶ Потоки отнимают много ресурсов процессора
- ▶ Часто запутывают логику



# План урока

- ▶ Проблемы многопоточности
- ▶ **Реактивное введение в корутины**
- ▶ DSL – это не модем

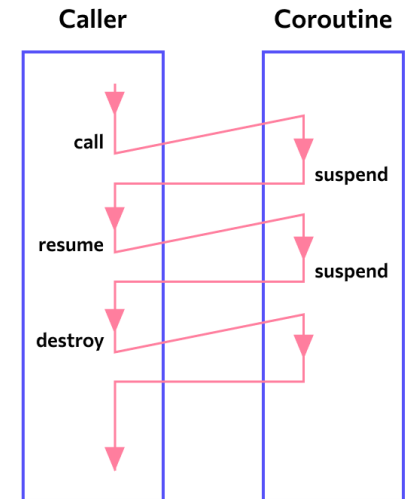
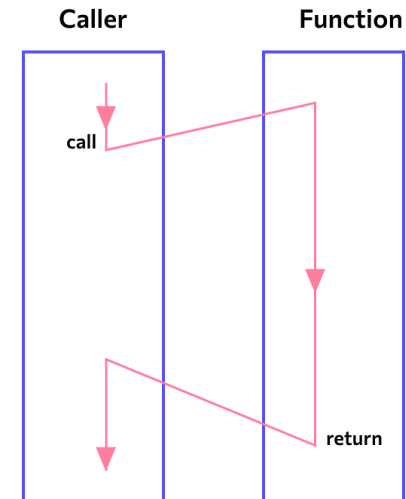
# Корутины

## Сопрограммы

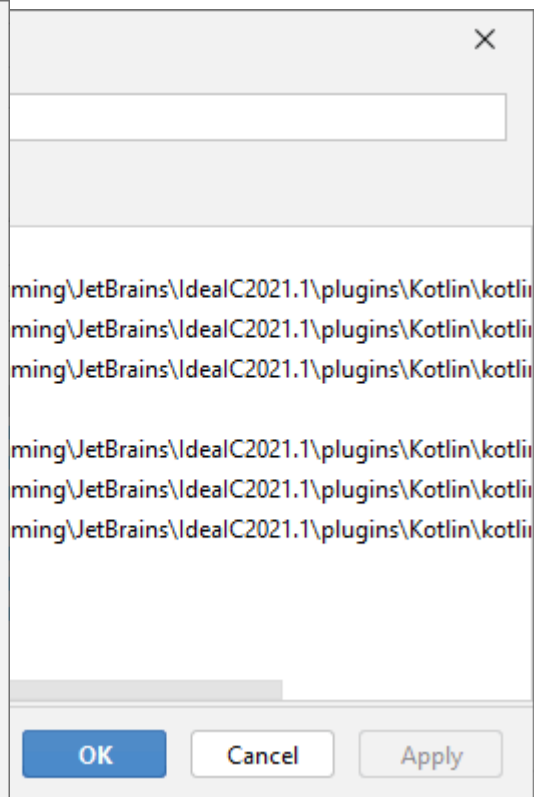
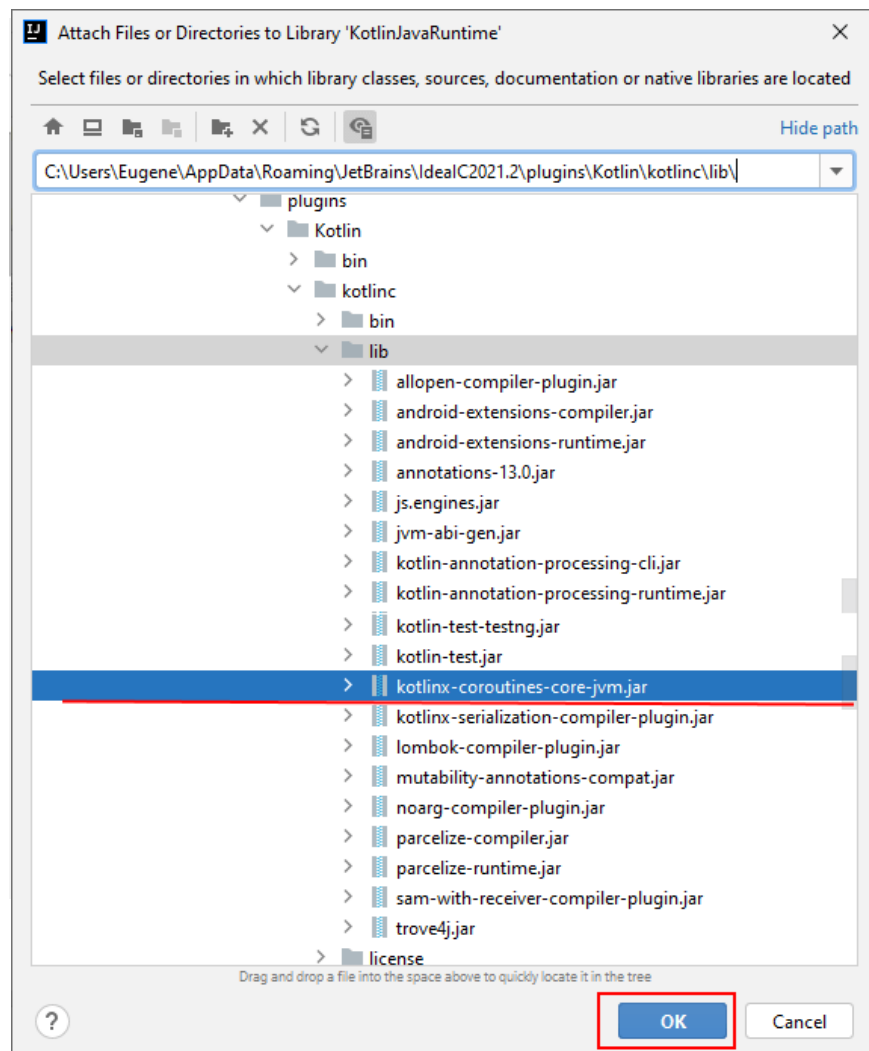
это потоки исполнения кода, которые организуются поверх системных потоков

- ▶ работают на более высоком уровне: несколько сопрограмм могут поочередно выполнять свой код на одном системном потоке.
- ▶ Приостанавливаются и возобновляют работу больше одного раза.
- ▶ Действуют по стратегии LIFO
- ▶ Меньший расход ресурсов на создание
- ▶ Гибкое управление (переключаться между корутинами можно вручную)

**kotlinx.coroutines.**



# Поддержка корутин



# Пример определения корутин

**suspend** - функция, которая может приостановить свое выполнение и возобновить его через некоторый период времени

```
suspend fun main() {  
    for (i in 0..5) {  
        //функция из пакета coroutines  
        //задержка выполнения в мс.  
        delay(400L)  
        println(i)      0 1 2 3 4 5 Hello Coroutines  
    }  
    println("Hello Coroutines")  
}
```

- ▶ **coroutineScope()** - создает контекст корутины
  - ▶ ожидает выполнения всех внутренних корутин
  - ▶ можно применить только к suspend функции
- ▶ **launch** – запуск билдера корутины, не возвращающей результаты

```
suspend fun main() = coroutineScope{  
    launch{  
        for(i in 0..5){  
            delay(400L)  
            println(i)  
        }  
    }  
    println("Hello Coroutines")  
}
```

# Область корутины

- ▶ **coroutineScope()** - приостанавливает выполнение вызывающего потока, освобождая для использования другими ресурсами
- ▶ **runBlocking()** - блокирует вызывающий поток, пока все корутины не завершат выполнение

```
class CountHolder() {
    companion object {
        //статическое поле
        var count = 0
    }
}

suspend fun main() {
    val outer = "outer"
    val inner = "inner"
    //создание контекста
    coroutineScope {
        launch{
            //вызываем инкрементер с корутином
            CountHolder.incrementer()
            coroutinePrint(inner)
        }
        launch {
            CountHolder.incrementer()
            coroutinePrint("$inner")
        }
    }
    println("Hello from $outer")
    CountHolder.incrementer()
    println("final: ${CountHolder.count}")//299, 300
}

fun coroutinePrint(type: String) {
    println("Hello from $type coroutine!")
}

fun CountHolder.Companion.incrementer() = runBlocking {
    launch {
        for (i in 1..100) {
            count++
            println(count)
        }
    }
}
```

# Coroutine builder. Launch

- ▶ **launch()** - применяется, когда нам не надо возвращать результат из корутины
  - ▶ возвращает объект **Job**, с помощью которого можно
    - управлять запущенной корутиной
  - ▶ **launch(start = CoroutineStart.LAZY)** – отложенный старт
  - ▶ **job.join()** – ожидаем завершения корутины
  - ▶ **job.cancel()** - отмена выполнения
    - ▶ **cancelAndJoin()** – отменяем и ждём завершения

```
suspend fun main() = coroutineScope{  
    val job = launch{  
        for(i in 1..5){  
            println(i)  
            delay(400L)  
        }  
    }  
    println("Start")  
    // ожидаем завершения корутины  
    job.join()  
    println("End")  
}
```

# Coroutine builder. Async

- ▶ **async()** - когда требуется вернуть результат из корутины
  - ▶ возвращает объект **Deferred**, который ожидает получения результата корутины
  - ▶ для получения результата используется функция **await()**
  - ▶ отложенный старт срабатывает на **await()** и **start()**

```
suspend fun main() = coroutineScope{  
    val numDeferred1 = async{ sum(1, 2)}  
    val numDeferred2 = async{ sum(3, 4)}  
    val numDeferred3 = async{ sum(5, 6)}  
    val num1 = numDeferred1.await()  
    val num2 = numDeferred2.await()  
    val num3 = numDeferred3.await()  
    println("number1: $num1  
            number2: $num2 number3: $num3")  
}  
  
suspend fun sum(a: Int, b: Int) : Int{  
    delay(500L)  
    return a + b  
}
```

# План урока

- ▶ Проблемы многопоточности
- ▶ Реактивное введение в корутины
- ▶ **DSL – это не модем**

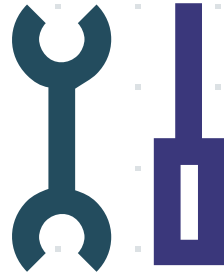


# Что такое DSL?

domain-specific language

это языки программирования с более высоким уровнем абстракции, которые отражают специфику решаемых с их помощью задач

универсальные языки



java, kotlin, c++, ruby,  
js...

предметно-ориентированные



SQL, HTML, CSS, Gradle,  
JSON...

# Зачем нужен DSL?

Часто ли вам приходилось снова и снова описывать стандартные экземпляры сущностей для довольно крупной модели данных?

```
public void someTest() {  
    Player player = new Player();  
    player.setHp(100);  
    player.setLevel(1);  
    /* ... */  
    NPC blacksmith = new NPC();  
    /* аналогично */  
}  
/* и т.д. */
```

```
game {  
  data  
  {  
    title("My Cool Game")  
    subject("Kill Dragon or smth")  
    player {  
      name = "sir Alx"  
      hp = 100  
    }  
    npc {  
      name = "sir RandomGuard"  
      hp = 120  
    }  
    npc {  
      name = "Dragon"  
      hp = 1000  
    }  
  }  
}
```

# Как использовать?

| Название функциональности      | DSL синтаксис                                                                                       | Обычный синтаксис                                                   |
|--------------------------------|-----------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|
| Переопределение операторов     | <code>collection += element</code>                                                                  | <code>collection.add(element)</code>                                |
| Псевдонимы типа                | <code>typealias Point = Pair&lt;Int, Int&gt;</code>                                                 | Создание пустых классов-наследников и прочие костыли                |
| Соглашение для get/set методов | <code>map["key"] = "value"</code>                                                                   | <code>map.put("key", "value")</code>                                |
| Мульти-декларации              | <code>val (x, y) = Point(0, 0)</code>                                                               | <code>val p = Point(0, 0); val x = p.first; val y = p.second</code> |
| Лямбда за скобками             | <code>list.forEach { ... }</code>                                                                   | <code>list.forEach({...})</code>                                    |
| Extention функции              | <code>mylist.first();</code> // метод first() отсутствует в классе коллекции<br><code>mylist</code> | Утилитные функции                                                   |
| Infix функции                  | <code>1 to "one"</code>                                                                             | <code>1.to("one")</code>                                            |
| Лямбда с обработчиком          | <code>Person().apply { name = «John» }</code>                                                       | Нет                                                                 |
| Контролирование контекста      | <code>@DslMarker</code>                                                                             | Нет                                                                 |

## Пример

```
game {  
    data { ... } }
```



```
game.invoke({ this.data({ }) })
```



```
operator fun invoke(init: GameContext.() -> Unit) {  
    GameContext().init() }
```

# Итог

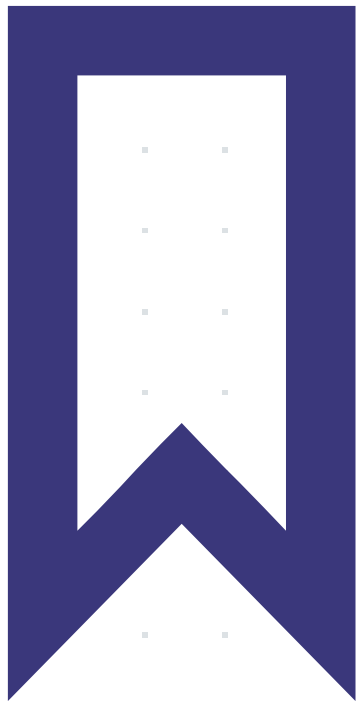
- ❑ Корутины – мощная штука, но сложная
  - ❑ выполняются поверх потоков
  - ❑ гораздо дешевле
  - ❑ иногда даже читабельнее
  - ❑ их надо контролировать
- ❑ В Kotlin удобно создавать DSL...
  - ❑ из-за синтаксиса лямбд
  - ❑ благодаря «сахару»



# Q&A



## Ещё по этой теме



### Ресурсы:

- ▶ <https://kotlinlang.ru/> - русскоязычная документация
- ▶ <https://play.kotlinlang.org/> - котлин онлайн
- ▶ <https://compscicenter.ru/courses/kotlinprogramming/>  
- курс Kotlin от CS center
- ▶ <https://kotlinlang.org/> - англоязычная документация
- ▶ <https://blog.skillfactory.ru/glossary/coroutine/> - про корутины
- ▶ [https://www.youtube.com/watch?v=fflVVWHpups&ab\\_channel=JUG.ru](https://www.youtube.com/watch?v=fflVVWHpups&ab_channel=JUG.ru) - Андрей  
Бреслав про корутины
- ▶ <https://habr.com/ru/company/haulmont/blog/341402/> - DSL с примерами

# Спасибо!



Aleksandr.Mityasov



@mityasov



Aleksandr.Mityasov@nexign.com

В настоящем документе содержится не предназначенная для публикации конфиденциальная информация и материалы, принадлежащие АО «Нэксайн» или его аффилированным лицам, охраняемые российским и международным законодательством об авторском, патентном праве, праве на секрет производства (ноу-хау), о товарных знаках и иных средствах индивидуализации.

Программное обеспечение АО «Нэксайн» и связанная с ним документация могут распространяться и использоваться только на основании отдельного лицензионного соглашения с АО «Нэксайн».

Любое использование, копирование, воспроизведение, публикация, обнародование, передача, изменение, переработка или перевод настоящего документа и содержащихся в нем информации и материалов, полностью или частично, в любой форме и любыми средствами без предварительного письменного разрешения АО «Нэксайн» строго запрещено.

Если настоящий документ предоставляется с программным обеспечением, лицензированным АО «Нэксайн», информация в нем предоставляется в соответствии с условиями гарантии, предоставляемой с лицензией на программное обеспечение.

Если настоящий документ не предоставляется с лицензионным программным обеспечением, информация в нем предоставляется «как есть» без каких-либо гарантий.

В обоих случаях АО «Нэксайн» не предоставляет никаких иных гарантий в отношении настоящего документа и содержащихся в нем информации и материалов, явных или подразумеваемых, включая, помимо прочего, подразумеваемые гарантии в отношении их качества, коммерческой пригодности, пригодности для определенной цели, ненарушения прав третьих лиц.

АО «Нэксайн» или его аффилированные лица ни при каких обстоятельствах не несут ответственность за любые прямые или косвенные убытки, случайные расходы или штрафы, возникшие в результате использования или невозможности использования материалов и/или информации, содержащихся в настоящем документе.

АО «Нэксайн» принимает надлежащие меры для обеспечения качества информации и материалов, содержащихся в настоящем документе. Однако документ может содержать технические неточности или опечатки, отсутствие ошибок не гарантируется.

Настоящий документ может быть изменен или дополнен без какого-либо уведомления. АО «Нэксайн» не обязуется обновлять настоящий документ на основе изменений в продуктах или услугах, как собственных, так и третьих лиц. АО «Нэксайн» может вносить улучшения или изменения в продукты или услуги, описанные в настоящем документе, в любое время без предварительного уведомления.